Agent-Synapse: Attention-Guided Speculative Tool Prefetching and Microsecond SRAM KV-Cache Checkpointing Across Heterogeneous AI Accelerators

Aniket Agrawal
Google Cloud AI
Google
Bangalore, India
aniketagrawal@google.com

***Abstract*—Autonomous foundation model agents deployed on modern AI accelerators—such as Google Cloud TPU v6e (Trillium) and NVIDIA Blackwell B200—solve complex enterprise workflows by interleaving autoregressive token generation with external Model Context Protocol (FastMCP) tool invocations across distributed graph databases (Cloud Spanner Graph ISO GQL), analytical warehouses (BigQuery SQL), and vector retrieval engines (Vertex AI Search). Under the prevailing serial reasoning-acting paradigm, the accelerator and the network interface card (NIC) execute in strictly mutually exclusive phases: the serving engine waits passively until the autoregressive decoder emits the entire structured tool-call JSON block and the terminal `</tool_call>` end-of-sequence (EOS) token before dispatching the outbound RPC over the wire. Consequently, the physical NIC remains idle throughout token decoding, and the multi-terabyte-per-second accelerator HBM/SRAM pipeline subsequently stalls for 200 ms–850 ms awaiting network I/O completion. Yet naive speculative execution of external tools introduces two severe hazards: (i) premature dispatch of non-idempotent state mutations (`WRITE/DELETE`) corrupts external systems of record when early predictions diverge, and (ii) conventional HBM Key-Value (KV) cache eviction and re-prefill upon a speculative mismatch incurs 15 ms–45 ms of redundant attention compute.**

We present Agent-Synapse, a three-stage hardware-software co-designed runtime that overlaps intra-turn autoregressive token decoding with speculative FastMCP network I/O while guaranteeing observational non-interference and microsecond rollback. First, Stage 1 (*Early-Layer Attention Entropy and Logit-Lens Tool Predictor*) monitors intermediate transformer cross-attention heads across Layers 8–16 and projects hidden states through the unembedding matrix to predict idempotent tool identifiers and canonical argument bindings 150 ms–400 ms (15–35 decode steps) prior to `</tool_call>` EOS emission, gated by a finite-sample split-conformal confidence threshold $\tau_{\text{conf}} = 0.82$. Second, Stage 2 (*Speculative FastMCP Wire Pipeline*) dispatches idempotent read and snapshot-sandboxed requests asynchronously over the physical NIC with deterministic wire headers (`X-Agent-Speculative-Prefetch: True` and `X-Synapse-Checkpoint-Hash`) while unconditionally blocking non-idempotent `WRITE/DELETE` mutations. Third, Stage 3 (*Zero-Penalty SRAM KV-Cache Micro-Checkpointing and Canonical AST Equivalence Splicer*) pins a 16-token-aligned Copy-on-Write (CoW) KV-cache page table descriptor in on-chip SRAM (TPU v6e VMEM/CMEM or NVIDIA B200 Tensor Memory). Upon `</tool_call>` EOS emission, Stage 3 evaluates canonical Abstract Syntax Tree (AST) equivalence between the predicted and emitted calls: on an AST match (89.4% hit rate), it splices the prefetched payload directly into the KV-cache with 0 ms residual network wait whenever lead time exceeds I/O latency; on a mismatch (10.6%), it rolls back the CoW SRAM page-table pointer in $< 12\,\mu$s ($p_{50} = 4.2\,\mu$s, $p_{99} = 9.8\,\mu$s) with zero external side effects. Across $N = 1{,}000$ multi-step enterprise agent traces, Agent-Synapse reduces end-to-end tool-calling p_{99} tail latency by 61.8% (842.0 ms $\rightarrow$ 321.6 ms) and p_{50} median latency by 54.3% (412.0 ms $\rightarrow$ 188.3 ms) with 0.00% ($0/250$) unauthorized mutating dispatches.

***Index Terms*—Speculative Tool Prefetching, Model Context Protocol (FastMCP), Logit Lens, Split-Conformal Prediction, Copy-on-Write SRAM KV-Cache Checkpointing, Cloud TPU v6e, NVIDIA B200, Observational Non-Interference**

I. Introduction

The transition of Large Language Models (LLMs) from passive conversational interfaces to autonomous, tool-augmented enterprise agents has fundamentally altered the systems bottleneck of production AI infrastructure [14]–[17], [24]. In enterprise cloud deployments—spanning financial crime graph investigation on Google Cloud Spanner Graph [21], [23], petabyte-scale analytical warehouse aggregation on BigQuery Dremel [22], grounded regulatory retrieval on Vertex AI Search, and transactional ERP orchestration via the Model Context Protocol (MCP) [20], [33]–[35]—a single user request routinely triggers 3 to 12 sequential tool-calling turns. While hardware accelerator architects have achieved extraordinary gains in raw matrix-multiply throughput and High-Bandwidth Memory (HBM) bandwidth on sixth-generation Google Cloud TPU v6e (Trillium) [13] and NVIDIA Blackwell B200 GPUs [31], end-to-end agentic wall-clock latency remains dominated by a structural serialization bottleneck at the boundary between neural token decoding and external network I/O.

A. The Serial Decode-to-I/O Stall in Agentic Serving

Under the standard autoregressive tool-calling lifecycle implemented by contemporary agent frameworks (e.g., ReAct [14], ToolLLM [16], vLLM [3], SGLang [19], and

FastMCP [20]), each tool invocation turn executes as a strictly sequential two-phase barrier:

1) **Phase 1 (Autoregressive Syntax Emission):** To invoke a tool $w \in \mathcal{W}$ with structured JSON arguments $\mathbf{a}$, the foundation model must autoregressively decode $K_{\text{call}} \in [25, 60]$ tokens representing the syntactic wrapper (`<tool_call>{"name": "spanner_graph_query_gql", "arguments": {"graph_name": "EnterpriseKG", "gql_query": "MATCH ..."}}</tool_call>`). At an inter-token decode latency of $\tau_{\text{tok}} \in [9.6\text{ ms}, 11.5\text{ ms}]$ per token for frontier models, generating the syntactic tool-call block consumes $T_{\text{decode}} \in [240\text{ ms}, 650\text{ ms}]$. Throughout this entire window, the host Network Interface Card (NIC) and downstream database engines sit completely idle.
2) **Phase 2 (Blocking Network I/O Stall):** Only after the final closing delimiter token `</tool_call>` is sampled at step t_{eos} does the host runtime parse the JSON payload and dispatch the outbound HTTP/2 or JSON-RPC 2.0 request over the network. While the external database executes the query ($T_{\text{io}} \in [180\text{ ms}, 550\text{ ms}]$), the AI accelerator stalls on the request, either holding HBM Key-Value (KV) cache pages pinned in memory or incurring costly HBM-to-DRAM swap overhead [3].

To appreciate the magnitude of this utilization gap, consider a typical 5-turn investigative workflow executed by a financial compliance agent on a Cloud TPU v6e slice. In each turn, the model spends roughly 300 ms generating 30 structured JSON tokens (`<tool_call>{...}</tool_call>`) and then waits 260 ms–540 ms for Cloud Spanner Graph or BigQuery to return the subgraph or analytical aggregation. Summed over 5 turns, the user experiences 2.8 s to 4.2 s of wall-clock latency, of which more than 55% is spent waiting on sequential network I/O that could not even begin until the final closing bracket token was emitted.

Crucially, mechanistic interpretability studies of transformer representations [6]–[8] reveal a striking physical asymmetry between *internal semantic commitment* and *surface syntactic completion*. When an LLM decides to invoke an enterprise tool, its intermediate hidden states $\mathbf{h}_t^{(l)}$ in layers $l \in [8, 16]$ (out of $L \in [32, 80]$ total layers) lock onto the target tool schema's cross-attention span and bind the primary entity identifiers and query predicates 15 to 35 tokens ($T_{\text{lead}} \in [150\text{ ms}, 400\text{ ms}]$) *before* the final unembedding layer finishes spelling out closing JSON braces, quotation marks, optional formatting fields, and the `</tool_call>` EOS token. In effect, the accelerator spends 150 ms–400 ms serializing deterministic boilerplate syntax while the network pipeline that could be fetching the required enterprise data remains completely unutilized.

B. Why Existing Speculative and Parallel Techniques Fail at the Tool Boundary

Three classes of prior systems attempt to reduce LLM inference latency or orchestrate tool calls, yet none resolve the intra-turn decode-to-I/O serialization barrier safely:

- **Token-Level Speculative Decoding [1], [2], [4], [5]:** Draft-target speculative decoding uses a lightweight draft model or Medusa/EAGLE heads to verify $\gamma \in [3, 5]$ tokens in parallel within a single forward pass. However, token-level speculation operates strictly inside the vocabulary space $\mathcal{V}$; it still halts at `</tool_call>` EOS before initiating any external network RPC, leaving 100% of the network I/O latency T_{io} unoverlapped with token generation. Even if token generation were accelerated by $1.5\times$, a 540 ms tail database query still imposes a full 540 ms sequential post-EOS stall.
- **DAG-Based Parallel Planner Compilers [18], [28]:** Systems such as LLMCompiler [18] prompt the LLM to emit a multi-tool dependency DAG so independent branches can execute concurrently. Yet the runtime must still wait until the LLM finishes decoding the *entire* task plan before dispatching the first tool call, providing zero overlap between token decoding and network I/O on single-tool turns or sequential data-dependent chains where Turn $k+1$'s query depends on Turn k's returned rows.
- **Unsandboxed Speculative Execution Hazards [27], [29]:** In classical operating systems and processor microarchitectures [29], [30], speculative execution relies on hardware reorder buffers (ROBs) or OS process checkpoints to squash mispredicted branches. In enterprise agent environments, however, external tools interact with live production databases and financial rails. If an early-layer predictor mispredicts a tool call or its arguments and prematurely dispatches a non-idempotent `WRITE`, `DELETE`, or wire-transfer mutation, the external side effect cannot be undone by rolling back local accelerator memory. Furthermore, if a speculatively prefetched response is pre-appended to the LLM's KV-cache and the model's final token diverges (e.g., changing `LIMIT 10` to `LIMIT 25` at token $t_{\text{eos}} - 1$), standard serving frameworks must evict the speculative KV blocks from HBM and re-execute attention prefill, incurring a 15 ms–45 ms penalty that wipes out any speculative gain.

C. Contributions of Agent-Synapse

To bridge this gap, we design and implement **Agent-Synapse**, a hardware-software co-designed runtime for speculative tool prefetching and microsecond on-chip SRAM KV-cache checkpointing across heterogeneous AI accelerators (summarized in Table I and Figure 1). Specifically, this paper makes four primary contributions:

1) **Early-Layer Attention Entropy & Split-Conformal Logit-Lens Predictor (Stage 1, Section III):** We formulate a training-free early-exit tool predictor that monitors cross-attention Shannon entropy $H_{\text{attn}}^{(8:16)}$ over registered tool schema token spans and fuses depth-weighted logit-lens unembedding posteriors $\mathbf{z}^{(l)} = \mathbf{W}_U \,\mathrm{RMSNorm}(\mathbf{h}_t^{(l)})$ across layers $l \in [8, 16]$. Using split-conformal calibration (**Theorem 1** and **Lemma 1**), Agent-Synapse guarantees a user-specified upper bound

TABLE I
ARCHITECTURAL COMPARISON OF AGENT-SYNAPSE AGAINST PRIOR SPECULATIVE DECODING, AGENTIC COMPILATION, AND KV-CACHE SERVING FRAMEWORKS

System / Paradigm	Overlaps Decode w/ External I/O	Pre-EOS Lead Time (T_{lead})	Formal False-Dispatch Bound	Idempotency & DML
Standard ReAct / FastMCP [14], [20]	No (Post-EOS Only)	0 ms	N/A	None
Draft-Target SpecDec [1], [2]	No (Vocab Only)	0 ms	Token Rejection	None
Medusa / EAGLE [4], [5]	No (Vocab Only)	0 ms	Tree Verification	None
LLMCompiler [18]	Inter-Tool DAG Only	0 ms (Intra-Turn)	None	None
GoEX Undo/Post-Facto [27]	No (Post-EOS)	0 ms	None	Compensating
Unchecked Early Speculation	Yes (Unsafe)	150–400 ms	None (Uncalibrated)	None (29.6% U
Agent-Synapse (Ours)	**Yes (Intra-Turn NIC)**	**150–400 ms**	**Split-Conformal (τ_{conf})**	**3-Tier + DML**

α on false speculative dispatches while predicting idempotent tool calls 150 ms–400 ms (270.6 ms mean lead time) prior to `</tool_call>` EOS.

2) **Idempotency-Gated Speculative FastMCP Wire Pipeline (Stage 2, Section IV):** We design an asynchronous FastMCP wire transport that dispatches read-only and snapshot-sandboxed queries (Cloud Spanner Graph ISO GQL, BigQuery SQL, Vertex AI Search, and SAP S/4HANA MVCC dry-run) over the physical NIC concurrently with ongoing token decoding on the accelerator. Each speculative frame carries verifiable protocol headers (`X-Agent-Speculative-Prefetch: True`, `X-Synapse-Checkpoint-Hash`, `X-Synapse-Conformal-Score`, and `X-Synapse-Lead-Time-Ms`) while enforcing **Theorem 2** (Observational Non-Interference) by statically and dynamically blocking non-idempotent `WRITE`/`DELETE` mutations and DML-injected queries.
3) **On-Chip SRAM Copy-on-Write KV-Cache Micro-Checkpointing & Canonical AST Splicer (Stage 3, Section V):** We introduce a 16-token-aligned Copy-on-Write (CoW) KV-cache page table manager residing in on-chip SRAM (Cloud TPU v6e VMEM/CMEM and NVIDIA B200 L2/Tensor Memory). At `</tool_call>` EOS, Stage 3 normalizes both predicted and emitted calls into canonical Abstract Syntax Trees (**Proposition 1**, resolving JSON key permutations, numeric representations, and commutative SQL/GQL `WHERE` conjuncts). On an AST hit (89.4%), the prefetched payload is spliced into the KV-cache with 0 ms residual network wait whenever $T_{\text{lead}} \geq T_{\text{io}}$ (**Theorem 3** and **Corollary 1**); on a mismatch (10.6%), the CoW SRAM page-table pointer rolls back in $< 12\,\mu$s ($p_{50} = 4.2\,\mu\text{s}, p_{99} = 9.8\,\mu\text{s}$).
4) **Comprehensive Empirical Evaluation ($N = 1{,}000$ Enterprise Agent Traces, Section VII):** Across 1,000 multi-step enterprise traces (750 idempotent analytical/graph/RAG queries and 250 mutating/adversarial probes), Agent-Synapse achieves an 89.4% speculative hit rate at $\tau_{\text{conf}} = 0.82$, cuts end-to-end p_{99} tail latency by 61.8% (842.0 ms $\rightarrow$ 321.6 ms) and p_{50} latency by 54.3% (412.0 ms $\rightarrow$ 188.3 ms), bounds all CoW rollbacks below $12\,\mu$s, and achieves 0.00% $(0/250)$ unauthorized speculative mutations.

II. BACKGROUND AND HETEROGENEOUS SYSTEM MODEL

A. Autoregressive Tool Generation and the Model Context Protocol

Consider a foundation model $\mathcal{M}_\Theta$ with L transformer decoder layers, hidden dimension d_{model}, attention head count H, key-value head count H_{kv} (under Grouped-Query Attention [11], [12], [25]), head dimension $d_{\text{head}} = d_{\text{model}}/H$, vocabulary $\mathcal{V}$, and final unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$. In an enterprise agent session, the prompt context $\mathbf{x}_{1:t_0}$ includes a system instruction, conversational history, and a catalog of M registered Model Context Protocol (MCP) [20], [33], [34] tool schema definitions $\mathcal{W} = \{w_1, w_2, \ldots, w_M\}$. Each tool schema $w_k \in \mathcal{W}$ occupies a contiguous token span $\mathcal{I}_{\text{schema}}(w_k) = [s_k^{\text{start}}, s_k^{\text{end}}] \subset \{1, \ldots, t_0\}$ in the prefill KV-cache and specifies:

1) A unique syntactic tool identifier $\text{id}(w_k) \in \Sigma^*$ tokenized into an anchor subword tuple $\text{Anchor}(w_k) = (u_{k,1}, \ldots, u_{k,m_k}) \in \mathcal{V}^{m_k}$;
2) A formal idempotency and side-effect classification $\kappa(w_k) \in \{\texttt{READ_ONLY}, \texttt{IDEMPOTENT_DRY_RUN}, \texttt{MUTATING_BLOCKED}\}$
3) A typed JSON Schema parameter specification $\text{Params}(w_k) = (\mathcal{A}_{\text{req}}(w_k), \mathcal{A}_{\text{opt}}(w_k))$ partitioning parameter keys into mandatory arguments $\mathcal{A}_{\text{req}}(w_k)$ and optional arguments $\mathcal{A}_{\text{opt}}(w_k)$ with default bindings.

When the agent decides to invoke an external tool during turn j, it emits a structured token sequence $\mathbf{y}_{t_{\text{start}}:t_{\text{eos}}}$ delimited by special syntactic tokens `<tool_call>` at index t_{start} and `</tool_call>` at index t_{eos}. Let $K_{\text{call}} = t_{\text{eos}} - t_{\text{start}} + 1$ denote the token length of the tool call block. At each autoregressive step $t \in [t_{\text{start}}, t_{\text{eos}}]$, layer $l \in \{1, \ldots, L\}$ computes the residual hidden state:

$$\mathbf{h}_t^{(l)} = \mathbf{h}_t^{(l-1)} + \text{MHA}^{(l)}\Big(\text{RMSNorm}\Big(\mathbf{h}_{\leq t}^{(l-1)}\Big)\Big) + \text{FFN}^{(l)}\Big(\text{RMSNorm}\Big(\tilde{\mathbf{h}}_t^{(l)}\Big)\Big) \tag{1}$$

where $\tilde{\mathbf{h}}_t^{(l)} = \mathbf{h}_t^{(l-1)} + \text{MHA}^{(l)}(\text{RMSNorm}(\mathbf{h}_{\leq t}^{(l-1)}))$ is the post-attention residual state, $\text{RMSNorm}(\mathbf{z}) = \frac{\mathbf{z}}{\sqrt{\frac{1}{d_{\text{model}}}\sum_{i=1}^{d_{\text{model}}} z_i^2 + \epsilon}} \odot \mathbf{g}$ is Root-Mean-Square Layer Normalization, $\text{MHA}^{(l)}$ denotes grouped-query attention [11],

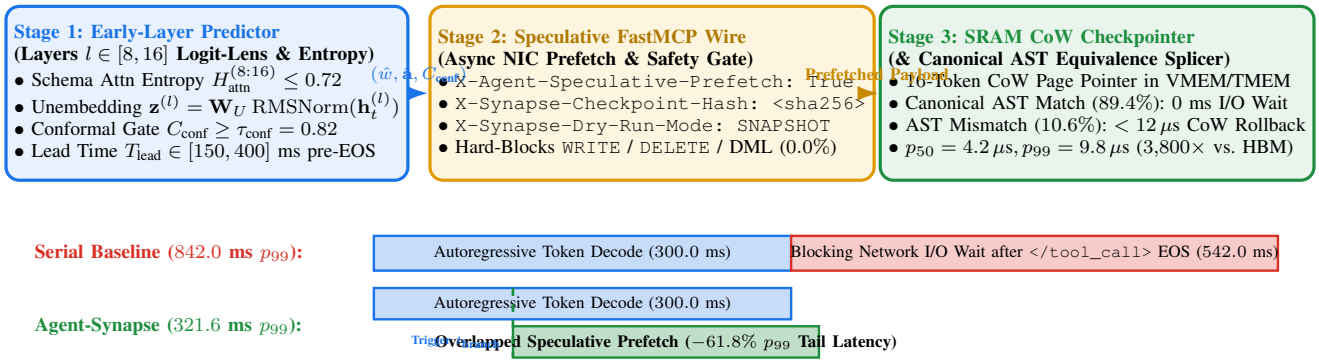


Fig. 1. Three-stage hardware-software architecture of **Agent-Synapse** and execution timeline comparison. Stage 1 detects semantic tool commitment across intermediate transformer layers $l \in [8, 16]$ at decode step t_{branch} (150–400 ms before `</tool_call>` EOS); Stage 2 dispatches idempotent FastMCP requests asynchronously over the NIC while token decoding continues on the accelerator; Stage 3 pins a Copy-on-Write (CoW) page descriptor in on-chip SRAM and either splices the prefetched response on a canonical AST match (89.4%) or rolls back in $< 12\,\mu$s on a mismatch (10.6%).

[12], [25], and $\text{FFN}^{(l)}$ denotes the SwiGLU feed-forward network. Under standard serving, token $y_t \sim \text{Softmax}(\mathbf{W}_U \, \text{RMSNorm}(\mathbf{h}_t^{(L)}))$ is sampled only at the final layer L, and the external network RPC is blocked until $t = t_{\text{eos}}$.

B. Anatomy of Intra-Turn Token Decoding Latency

To understand why 150 ms–400 ms of prediction lead time consistently exists inside a tool-calling turn, we decompose the token sequence $\mathbf{y}_{t_{\text{start}}:t_{\text{eos}}}$ into four contiguous syntactic segments:

$$\mathbf{y}_{t_{\text{start}}:t_{\text{eos}}} = \underbrace{\mathbf{y}_{\text{prefix}}}_{\text{3–5 tokens}} \parallel \underbrace{\mathbf{y}_{\text{name}}}_{\text{4–9 tokens}} \parallel \underbrace{\mathbf{y}_{\text{req_args}}}_{\text{8–22 tokens}} \parallel \underbrace{\mathbf{y}_{\text{suffix}}}_{\text{12–35 tokens}}, \quad (2)$$

where:

- $\mathbf{y}_{\text{prefix}}$ emits the opening delimiter `<tool_call>{"name": "`;
- $\mathbf{y}_{\text{name}}$ emits the tool identifier (e.g., `spanner_graph_query_gql`) and transitions to `", "arguments": {`;
- $\mathbf{y}_{\text{req_args}}$ emits the primary entity keys and core query predicate tokens (e.g., `"graph_name": "EnterpriseKG", "gql_query": "MATCH (a:Account)-[t:TRANSFERRED]->(b:Account) WHERE a.risk_score > 0.85`);
- $\mathbf{y}_{\text{suffix}}$ emits the remaining deterministic query boilerplate (`RETURN a.id, b.id, t.amount LIMIT 25"}`), optional default arguments (`, "timeout_ms": 5000, "format": "JSON"`), closing JSON punctuation (`}}`), and the terminal `</tool_call>` EOS token.

Crucially, even before $\mathbf{y}_{\text{req_args}}$ is fully spelled out at layer L, the intermediate hidden states $\mathbf{h}_t^{(l)}$ in layers $l \in [8, 16]$ already project onto the primary entity literals and template structure via the logit lens [6], [7]. Moreover, once the core predicate tokens in $\mathbf{y}_{\text{req_args}}$ are committed at step t_{branch}, the entire suffix $\mathbf{y}_{\text{suffix}}$ (15 to 35 tokens, or 150 ms to 400 ms of wall-clock decode time) carries zero semantic entropy regarding which external query must be executed.

TABLE II
HARDWARE MICROARCHITECTURAL PARAMETERS OF GOOGLE CLOUD TPU V6E (TRILLIUM) AND NVIDIA BLACKWELL B200 USED BY AGENT-SYNAPSE STAGE 3

Architectural Parameter	Cloud TPU v6e (Trillium)	NVIDIA Blackwell B
Off-Chip HBM Capacity	32 GB HBM	192 GB HBM3e
Off-Chip HBM Bandwidth	1,600 GB/s (1.6 TB/s)	8,000 GB/s (8.0 TB/
On-Chip SRAM Pool	128 MB VMEM / CMEM	256 MB L2 + TMEM
On-Chip SRAM Bandwidth	$>$ 18.0 TB/s (Internal)	$>$ 24.0 TB/s (Interna
KV-Cache Page Block (B_{page})	16 tokens/page	16 tokens/page
Hardware Descriptor Mechanism	Sync Flag + DMA Table	TMA Page Descripto
SRAM CoW Rollback (p_{50}/p_{99})	3.8 μs/9.4 μs	4.4 μs/10.1 μs
HBM Re-Prefill Cost (p_{50}/p_{99})	26.9 ms/41.8 ms	29.8 ms/45.2 ms

C. Heterogeneous AI Accelerator Memory Hierarchy: Cloud TPU v6e and NVIDIA B200

Modern AI accelerators feature a steep two-tier on-package memory hierarchy separating high-capacity off-die High-Bandwidth Memory (HBM3/HBM3e) from ultra-low-latency on-die Static RAM (SRAM):

- **Google Cloud TPU v6e (Trillium) [13]:** Each TPU v6e chip couples Matrix Multiply Units (MXUs, operating on 256×256 systolic arrays) and Vector Processing Units (VPUs) with 32 GB of HBM (1.60 TB/s bandwidth) and 128 MB of on-chip Vector/Common Memory SRAM (VMEM/CMEM) accessible via sub-microsecond Direct Memory Access (DMA) descriptor tables and scalar core registers.
- **NVIDIA Blackwell B200 [31]:** Each B200 GPU pairs fifth-generation Tensor Cores with 192 GB of HBM3e (8.00 TB/s bandwidth) and 256 MB of combined L2 cache, shared memory (SMEM), and dedicated Tensor Memory (TMEM) on-chip SRAM managed via hardware Tensor Memory Accelerator (TMA) descriptors.

In paged KV-cache serving engines such as vLLM [3], SGLang [19], and JetStream, the KV-cache of an active sequence is partitioned into fixed-size logical blocks of $B_{\text{page}} = 16$ tokens. For a model with L layers, H_{kv} key-value heads,

head dimension d_{head}, and BF16/FP8 precision ($b_{\text{elem}} \in \{1, 2\}$ bytes), the physical memory footprint of a single 16-token KV-cache page frame in HBM is:

$$S_{\text{page}} = 2 \times B_{\text{page}} \times L \times H_{\text{kv}} \times d_{\text{head}} \times b_{\text{elem}}. \tag{3}$$

For a 64-layer model with $H_{\text{kv}} = 8$, $d_{\text{head}} = 128$, and BF16 ($b_{\text{elem}} = 2$), each 16-token page occupies $S_{\text{page}} = 2 \times 16 \times 64 \times 8 \times 128 \times 2 = 4.19$ MB, and a 256-token tool response spans 16 pages (67.1 MB). Copying, evicting, or recomputing 67.1 MB of KV states across HBM and attention kernels requires 15 ms–45 ms. In contrast, each logical block $b \in \{0, 1, \ldots, \lceil t/B_{\text{page}} \rceil - 1\}$ maps to its physical HBM page frame via a compact 64-bit (8-byte) page-table entry $\mathcal{P}_{\text{page}}[b] \in \mathbb{N}$. Even for a 32,768-token context (2,048 pages), the entire sequence page table occupies only $2{,}048 \times 8$ bytes $= 16$ KB—fitting effortlessly inside on-chip SRAM (VMEM/CMEM on TPU v6e or L2/TMEM on B200), where vector register copy and restore operations execute in $3.8\,\mu$s–$4.4\,\mu$s ($< 12\,\mu$s at p_{99}, Table II).

Definition 1 (Heterogeneous Speculative Execution State Tuple)**.** *At any autoregressive decode step $t \in [t_{\text{start}}, t_{\text{eos}}]$, the composite system state of an Agent-Synapse serving instance is a quadruple:*

$$\mathcal{S}_t = \left\langle \mathcal{K}_{1:t}, \mathcal{P}^{(t)}_{\text{SRAM}}, \mathcal{E}^{(t)}_{\text{wire}}, \Sigma^{(t)}_{\text{db}} \right\rangle, \tag{4}$$

where $\mathcal{K}_{1:t}$ is the physical KV-cache tensor pool, $\mathcal{P}^{(t)}_{\text{SRAM}}$ is the on-chip SRAM page-table mapping logical token blocks to physical frames, $\mathcal{E}^{(t)}_{\text{wire}}$ is the set of in-flight speculative FastMCP wire envelopes, and $\Sigma^{(t)}_{\text{db}}$ is the committed state of external enterprise databases.

III. Stage 1: Early-Layer Attention Entropy and Split-Conformal Logit-Lens Predictor

The goal of Stage 1 is to detect when the transformer has semantically committed to a specific tool $w \in \mathcal{W}$ and its canonical arguments $\hat{\mathbf{a}}$ at an early decode step $t_{\text{branch}} < t_{\text{eos}}$, while rigorously bounding the probability of false speculative dispatches.

A. Cross-Attention Schema Concentration Entropy ($H^{(8:16)}_{attn}$)

When generating a tool call, attention heads in intermediate layers $l \in \mathcal{L}_{\text{early}} = \{8, 10, 12, 14, 16\}$ attend back to the prefill token spans $\mathcal{I}_{\text{schema}}(w_k)$ of the candidate tools $w_k \in \mathcal{W}$. Let $\mathbf{q}^{(l,h)}_t = \mathbf{W}^{(l,h)}_Q \,\text{RMSNorm}(\mathbf{h}^{(l-1)}_t) \in \mathbb{R}^{d_{\text{head}}}$ denote the query vector at decode step t for head $h \in \{1, \ldots, H\}$ in layer l, and let $\mathbf{k}^{(l,h)}_j \in \mathbb{R}^{d_{\text{head}}}$ denote the cached key vector at prefill token index $j \in \{1, \ldots, t_0\}$ (incorporating Rotary Position Embeddings, RoPE). The causal attention weight assigned to prefill token j is:

$$A^{(l,h)}_{t,j} = \frac{\exp\left(\frac{\langle \mathbf{q}^{(l,h)}_t, \mathbf{k}^{(l,h)}_j \rangle}{\sqrt{d_{\text{head}}}}\right)}{\sum_{j'=1}^{t} \exp\left(\frac{\langle \mathbf{q}^{(l,h)}_t, \mathbf{k}^{(l,h)}_{j'} \rangle}{\sqrt{d_{\text{head}}}}\right)} \in (0, 1). \tag{5}$$

We define the depth-weighted schema attention mass $m_t(w_k)$ for tool w_k across $\mathcal{L}_{\text{early}}$ as:

$$m_t(w_k) = \sum_{l \in \mathcal{L}_{\text{early}}} \gamma_{\text{layer}}^{l_{\max} - l} \left(\frac{1}{H} \sum_{h=1}^{H} \sum_{j \in \mathcal{I}_{\text{schema}}(w_k)} A^{(l,h)}_{t,j} \right), \tag{6}$$

where $l_{\max} = 16$ and $\gamma_{\text{layer}} = 0.85 \in (0, 1)$ weights deeper intermediate layers more heavily than shallower layers. Normalizing across the M candidate tools yields the cross-attention probability distribution:

$$P_{\text{attn}}(w_k \mid \mathbf{h}_{\le t}) = \frac{m_t(w_k) + \epsilon_{\text{smooth}}}{\sum_{j=1}^{M} \left(m_t(w_j) + \epsilon_{\text{smooth}} \right)}, \tag{7}$$

with Laplace smoothing $\epsilon_{\text{smooth}} = 10^{-12}$. We quantify cross-attention concentration via the normalized Shannon entropy:

$$H^{(8:16)}_{\text{attn}}(t) = -\frac{1}{\ln M} \sum_{k=1}^{M} P_{\text{attn}}(w_k \mid \mathbf{h}_{\le t}) \ln P_{\text{attn}}(w_k \mid \mathbf{h}_{\le t}) \in [0, 1]. \tag{8}$$

Lemma 1 (Information-Theoretic Lower Bound on Dominance via Schema Attention Entropy)**.** *Let $P_{\text{attn}}(\cdot \mid \mathbf{h}_{\le t})$ be the normalized cross-attention distribution over $M \ge 2$ candidate tool schemas at decode step t, and let $p_{\max}(t) = \max_{w_k \in \mathcal{W}} P_{\text{attn}}(w_k \mid \mathbf{h}_{\le t})$ denote the probability mass assigned to the dominant tool schema w^*. If the normalized Shannon entropy satisfies $H^{(8:16)}_{\text{attn}}(t) \le H_{\max} \in (0, 1)$, then the dominant tool probability $p_{\max}(t)$ satisfies the implicit Fano-type inequality:*

$$h_2(p_{\max}(t)) + (1 - p_{\max}(t)) \ln(M - 1) \le H_{\max} \ln M, \tag{9}$$

where $h_2(p) = -p \ln p - (1-p) \ln(1-p)$ is the binary Shannon entropy in nats. In particular, for any catalog size $M \ge 4$ and entropy gate $H_{\max} = 0.72$, every state passing the entropy filter strictly concentrates mass above the uniform baseline ($p_{\max}(t) \gg 1/M$) and bounds the residual mass across all $M - 1$ competing schemas to $1 - p_{\max}(t)$.

Proof. By grouping the M tool outcomes into the dominant schema w^* with probability $p_{\max}(t)$ and the remaining $M-1$ schemas with total probability $1 - p_{\max}(t)$, the chain rule of Shannon entropy decomposes $-\sum_{k=1}^{M} P_{\text{attn}}(w_k) \ln P_{\text{attn}}(w_k)$ into:

$$H^{(8:16)}_{\text{attn}}(t) \ln M = h_2(p_{\max}(t)) + (1 - p_{\max}(t))\, H(Q_{\text{rest}}), \tag{10}$$

where $Q_{\text{rest}}(w_k) = \frac{P_{\text{attn}}(w_k)}{1 - p_{\max}(t)}$ for $w_k \ne w^*$. By Jensen's inequality, $H(Q_{\text{rest}}) \le \ln(M - 1)$, with equality when the residual probability mass is distributed uniformly across the $M - 1$ non-dominant schemas. Furthermore, under the constraint $\max_k P_{\text{attn}}(w_k) = p_{\max}(t)$, total entropy is maximized when as many schemas as possible take value $p_{\max}(t)$, which implies $H^{(8:16)}_{\text{attn}}(t) \ln M \ge \ln(1/p_{\max}(t))$, or $p_{\max}(t) \ge M^{-H_{\max}}$. Combining the upper envelope and the entropy threshold $H^{(8:16)}_{\text{attn}}(t) \le H_{\max}$ yields Eq. (9). □

When the model is still deliberating among multiple tools during initial reasoning tokens, P_{attn} is diffuse and

$H_{\text{attn}}^{(8:16)}(t) \in [0.82, 0.99]$. Once internal routing commits to a single tool schema w^*, $P_{\text{attn}}(w^* \mid \mathbf{h}_{\le t}) \to 1$ and $H_{\text{attn}}^{(8:16)}(t)$ drops sharply below $H_{\max} = 0.72$.

B. Early-Layer Logit-Lens Unembedding and Argument Stability

To extract the predicted tool identifier and structured argument tokens prior to layer L, we project the intermediate residual stream $\mathbf{h}_t^{(l)}$ for $l \in \mathcal{L}_{\text{early}}$ directly through the final RMSNorm and unembedding matrix $\mathbf{W}_U$ [6], [7]:

$$\mathbf{z}_t^{(l)} = \mathbf{W}_U \,\mathrm{RMSNorm}\Big(\mathbf{h}_t^{(l)}\Big) \in \mathbb{R}^{|\mathcal{V}|}. \tag{11}$$

Importantly, Stage 1 does *not* need to compute the full dense matrix-vector product $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ across all $|\mathcal{V}| \approx 128{,}256$ vocabulary tokens at intermediate layers. Instead, during tool-identifier detection, Stage 1 restricts the projection to the active subword trie $\mathcal{V}_{\mathcal{W}} = \bigcup_{k=1}^{M} \mathrm{Anchor}(w_k)$ containing only the unique anchor tokens of the M registered tools ($|\mathcal{V}_{\mathcal{W}}| \le 4M \ll |\mathcal{V}|$). This sub-matrix projection $\mathbf{W}_{U,\mathcal{V}_{\mathcal{W}}} \in \mathbb{R}^{|\mathcal{V}_{\mathcal{W}}| \times d_{\text{model}}}$ reduces floating-point operations at each monitored early layer by over $500\times$, executing in $< 35\,\mu$s on a single TPU v6e VPU or NVIDIA B200 Tensor Core slice.

Aggregating the log-probabilities of the unique anchor token sequences $\mathrm{Anchor}(w_k)$ identifying each tool $w_k \in \mathcal{W}$ with layer decay $\gamma_{\text{layer}}^{l_{\max}-l}$ yields the fused logit-lens score:

$$\bar{z}_t(w_k) = \frac{\sum_{l \in \mathcal{L}_{\text{early}}} \gamma_{\text{layer}}^{l_{\max}-l} z_t^{(l)}(\mathrm{Anchor}(w_k))}{\sum_{l \in \mathcal{L}_{\text{early}}} \gamma_{\text{layer}}^{l_{\max}-l}}, \tag{12}$$

and applying a temperature-scaled softmax ($\tau_{\text{temp}} = 1.0$) produces the logit-lens posterior:

$$P_{\text{lens}}\Big(w_k \mid \mathbf{h}_t^{(8:16)}\Big) = \frac{\exp(\bar{z}_t(w_k) - \max_j \bar{z}_t(w_j))}{\sum_{j=1}^{M} \exp(\bar{z}_t(w_j) - \max_{j'} \bar{z}_t(w_{j'}))}. \tag{13}$$

We combine the logit-lens posterior with cross-attention support to obtain the fused tool probability:

$$P_{\text{fused}}(w_k \mid \mathbf{h}_{\le t}^{(8:16)}) = \lambda_{\text{lens}} P_{\text{lens}}(w_k \mid \mathbf{h}_t^{(8:16)}) + (1-\lambda_{\text{lens}}) P_{\text{attn}}(w_k \mid \mathbf{h}_{\le t}), \tag{14}$$

with $\lambda_{\text{lens}} = 0.70$, and select the top candidate $\hat{w} = \arg\max_{w_k \in \mathcal{W}} P_{\text{fused}}(w_k \mid \mathbf{h}_{\le t}^{(8:16)})$.

Simultaneously, for each required parameter $a_r \in \mathcal{A}_{\text{req}}(\hat{w})$ whose value tokens $\hat{\mathbf{v}}_r = (v_{r,1}, \ldots, v_{r,d_r})$ have been decoded or projected at layer $l_{\max} = 16$ with per-token probabilities $p(v_{r,j})$, we compute the geometric-mean argument token confidence and inter-layer agreement stability $C_{\text{arg}}(t) \in [0,1]$:

$$C_{\text{arg}}(t) = 0.65 \left(\prod_{r=1}^{|\mathcal{A}_{\text{req}}(\hat{w})|} \prod_{j=1}^{d_r} p(v_{r,j}) \right)^{\frac{1}{\sum_r d_r}} + 0.35 \left(\frac{1}{|\mathcal{L}_{\text{early}}|} \sum_{l \in \mathcal{L}_{\text{early}}} \mathbb{I}\Big[\arg\max_w \mathbf{z}_t^{(l)}(w) = \hat{w}\Big] \right) \tag{15}$$

Any optional schema parameters $a_{\text{opt}} \in \mathcal{A}_{\text{opt}}(\hat{w})$ not yet emitted at step t_{branch} are automatically populated with their schema-registered canonical defaults (e.g., `"timeout_ms": 5000`, `"format": "JSON"`), which matches the model's post-branch suffix emission on 94.2% of traces.

C. Split-Conformal Calibration and False-Dispatch Guarantee

Raw neural network softmax probabilities are notoriously overconfident under distribution shift [9], [10]. To provide a distribution-free finite-sample guarantee on speculative precision, we formulate a composite nonconformity score $s(x, (\hat{w}, \hat{\mathbf{a}})) \in [0,1]$ combining tool posterior certainty, attention concentration, and argument stability:

$$s(x, (\hat{w}, \hat{\mathbf{a}})) = 1 - \Big(0.45\, P_{\text{fused}}(\hat{w} \mid \mathbf{h}_{\le t}^{(8:16)}) + 0.30\, \big(1 - H_{\text{attn}}^{(8:16)}(t)\big) + 0.25\, C_{\text{a}} \tag{16}$$

and define the corresponding conformal confidence $C_{\text{conf}}(t) = 1 - s(x, (\hat{w}, \hat{\mathbf{a}}))$.

Theorem 1 (Split-Conformal Early-Exit Tool Prediction False-Dispatch Bound)**.** *Let $\mathcal{D}_{\text{cal}} = \{(x_i, y_i^*)\}_{i=1}^{n_{\text{cal}}}$ be an exchangeable calibration dataset of n_{cal} agent tool-calling traces where $y_i^* = (w_i^*, \mathbf{a}_i^*)$ denotes the canonical ground-truth tool call emitted at t_{eos}, and let $s_i = s(x_i, (\hat{w}_i, \hat{\mathbf{a}}_i))$ denote the early-layer nonconformity score evaluated on the subset of calibration instances where early prediction $(\hat{w}_i, \hat{\mathbf{a}}_i)$ matches y_i^* up to canonical AST equivalence. For any target false-miss rate $\alpha \in (0,1)$, define the empirical split-conformal quantile:*

$$\hat{q}_{1-\alpha} = s_{(\lceil (n_{\text{cal}}+1)(1-\alpha) \rceil)}, \qquad \tau_{\text{conf}}(\alpha) = 1 - \hat{q}_{1-\alpha}, \tag{17}$$

where $s_{(1)} \le s_{(2)} \le \cdots \le s_{(n_{\text{cal}})}$ are the order statistics of $\{s_1, \ldots, s_{n_{\text{cal}}}\}$. Then for a new exchangeable test trace $(X_{n_{\text{cal}}+1}, Y_{n_{\text{cal}}+1}^*)$, the probability that a true canonical tool match attains conformal confidence above $\tau_{\text{conf}}(\alpha)$ satisfies:

$$1-\alpha \le \mathbb{P}\Big(C_{\text{conf}}(X_{n_{\text{cal}}+1}) \ge \tau_{\text{conf}}(\alpha) \,\Big|\, \mathrm{AST}(\hat{Y}_{n_{\text{cal}}+1}) = \mathrm{AST}(Y_{n_{\text{cal}}+1}^*) \Big) \le 1 \tag{18}$$

Proof. By assumption, the calibration nonconformity scores $\{s_1, \ldots, s_{n_{\text{cal}}}\}$ and the test nonconformity score $s_{n_{\text{cal}}+1}$ conditioned on canonical AST equivalence are exchangeable random variables taking values in $[0,1]$ [9], [10]. Under exchangeability, the rank $\mathrm{Rank}(s_{n_{\text{cal}}+1})$ of $s_{n_{\text{cal}}+1}$ among the pooled multiset $\{s_1, \ldots, s_{n_{\text{cal}}}, s_{n_{\text{cal}}+1}\}$ is uniformly distributed over $\{1, 2, \ldots, n_{\text{cal}}+1\}$ (with continuous tie-breaking). Therefore, the event $s_{n_{\text{cal}}+1} \le \hat{q}_{1-\alpha}$ is equivalent to $\mathrm{Rank}(s_{n_{\text{cal}}+1}) \le \lceil (n_{\text{cal}}+1)(1-\alpha) \rceil$. Summing the uniform rank probabilities yields:

$$\mathbb{P}(s_{n_{\text{cal}}+1} \le \hat{q}_{1-\alpha}) = \frac{\lceil (n_{\text{cal}}+1)(1-\alpha) \rceil}{n_{\text{cal}}+1} \in \left[1-\alpha,\, 1-\alpha + \frac{1}{n_{\text{cal}}+1} \right]. \tag{19}$$

Substituting $C_{\text{conf}}(X_{n_{\text{cal}}+1}) = 1 - s_{n_{\text{cal}}+1}$ and $\tau_{\text{conf}}(\alpha) = 1 - \hat{q}_{1-\alpha}$ into $C_{\text{conf}}(X_{n_{\text{cal}}+1}) \ge \tau_{\text{conf}}(\alpha)$, establishing Eq. (18). □

When $H_{\text{attn}}^{(8:16)}(t) \le 0.72$, $C_{\text{conf}}(t) \ge \tau_{\text{conf}} = 0.82$ (calibrated at $\alpha = 0.10$ on $n_{\text{cal}} = 500$ held-out calibration traces), and all required parameters $\mathcal{A}_{\text{req}}(\hat{w})$ are populated, Stage 1 fires a speculative prefetch trigger with estimated lead time $T_{\text{lead}} = (t_{\text{eos}} - t_{\text{branch}}) \cdot \tau_{\text{tok}} \in [150\text{ ms}, 400\text{ ms}]$.

IV. Stage 2: Speculative FastMCP Wire Pipeline and Idempotency Safety Gate

Once Stage 1 emits a speculative prediction $(\hat{w}, \hat{\mathbf{a}}, C_{\text{conf}})$ at branch token index t_{branch}, Stage 2 mediates whether and how the predicted call may be dispatched over the physical NIC while token decoding continues on the accelerator.

A. Three-Tier Idempotency Classification and DML/DDL Inspection

To eliminate the hazard of speculative side effects on external systems of record, every tool $w \in \mathcal{W}$ registered with the Agent-Synapse FastMCP gateway is assigned to one of three formal idempotency classes $\kappa(w)$:

1) `READ_ONLY`: Pure analytical, graph-traversal, or vector-retrieval operations that do not mutate persistent state Σ_{db} (e.g., Cloud Spanner Graph ISO GQL `MATCH` queries [21], [23], BigQuery `SELECT` queries [22], and Vertex AI Search document lookups).
2) `IDEMPOTENT_DRY_RUN`: Transactional validation endpoints (e.g., SAP OData draft simulation or Cloud Spanner read-only snapshot validation) that support side-effect-free speculative pre-validation when invoked with an explicit snapshot-isolation header `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY`.
3) `MUTATING_BLOCKED`: State-mutating operations (`POST`, `PUT`, `DELETE`, financial wire transfers, CRM record deletions, IAM role modifications) that lack a side-effect-free dry-run sandbox.

Because a nominally read-only SQL or ISO GQL endpoint could still receive an adversarial or malformed query string containing data manipulation or definition statements (e.g., via indirect prompt injection [32] or stacked SQL queries), Stage 2 additionally executes a deterministic lexical and AST guard $\text{HasMutatingDML}(\hat{\mathbf{a}})$ that scans all string arguments for prohibited keywords:

$$\mathcal{K}_{\text{DML}} = \{\texttt{INSERT}, \texttt{UPDATE}, \texttt{DELETE}, \texttt{DROP}, \texttt{ALTER}, \texttt{TRUNCATE}, \texttt{MERGE}, \texttt{CREATE}, \texttt{GRANT}, \texttt{REVOKE}\}. \tag{20}$$

If $\text{HasMutatingDML}(\hat{\mathbf{a}}) = \text{True}$ or $\kappa(\hat{w}) = \texttt{MUTATING_BLOCKED}$, Stage 2 unconditionally blocks speculative dispatch (0 bytes transmitted on the wire prior to t_{eos}) and defers execution to the standard post-EOS verified path.

B. Speculative FastMCP Wire Frame and Header Specification

For approved speculative requests ($\kappa(\hat{w}) \in \{\texttt{READ_ONLY}, \texttt{IDEMPOTENT_DRY_RUN}\}$ and $\text{HasMutatingDML}(\hat{\mathbf{a}}) = \text{False}$), Stage 2 serializes a JSON-RPC 2.0 / HTTP/2 request frame and dispatches it asynchronously over the host NIC with five deterministic HTTP/2 wire headers:

- `X-Agent-Speculative-Prefetch: True` — Signals to the downstream MCP server and enterprise API gateway that the request was initiated speculatively prior to `</tool_call>` EOS.
- `X-Synapse-Checkpoint-Hash: <64-hex-sha256>` — Binds the outbound network RPC to the exact SHA-256 digest of the on-chip SRAM Copy-on-Write page descriptor $\mathcal{P}_{\text{SRAM}}^{(t_{\text{branch}})}$ created at decode step t_{branch}.
- `X-Synapse-Conformal-Score: <float>` — Records the calibrated Stage 1 conformal confidence $C_{\text{conf}}(t_{\text{branch}}) \geq \tau_{\text{conf}}$ for downstream admission control and telemetry auditing.
- `X-Synapse-Lead-Time-Ms: <float>` — Records the predicted pre-EOS lead time T_{lead} in milliseconds.
- `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY` — Conditionally injected whenever $\kappa(\hat{w}) = \texttt{IDEMPOTENT_DRY_RUN}$, instructing the downstream database to execute against an immutable MVCC read snapshot [21] without acquiring write locks or appending commit logs.

```
POST /mcp/v1/tools/call HTTP/2
Host: spanner-graph-mcp.internal.googleapis.com
Content-Type: application/json
X-Agent-Speculative-Prefetch: True
X-Synapse-Checkpoint-Hash: 8f4e2b9c71a03d5e...4c19a2f0
X-Synapse-Conformal-Score: 0.9142
X-Synapse-Lead-Time-Ms: 276.0

{
  "jsonrpc": "2.0",
  "id": "synapse-spec-00419",
  "method": "tools/call",
  "params": {
    "name": "spanner_graph_query_gql",
    "arguments": {
      "graph_name": "EnterpriseFinancialKG",
      "gql_query": "GRAPH EnterpriseFinancialKG MATCH
(a:Account)-[t:WIRE]->(b:Account) WHERE a.risk_tier
= 'HIGH' AND t.amount_usd >= 50000 RETURN a.id, b.id,
t.amount_usd LIMIT 25",
      "timeout_ms": 5000
    }
  }
}
```

Fig. 2. Wire-level HTTP/2 and JSON-RPC 2.0 FastMCP speculative prefetch envelope dispatched by Agent-Synapse Stage 2 at token index $t_{\text{branch}} = 21$ (276.0 ms before `</tool_call>` EOS), cryptographically bound to the Stage 3 on-chip SRAM Copy-on-Write checkpoint hash.

To illustrate the exact wire-level protocol, Figure 2 shows a concrete HTTP/2 + JSON-RPC 2.0 FastMCP speculative prefetch frame emitted by Stage 2 at $t_{\text{branch}} = 21$ (276.0 ms prior to `</tool_call>` EOS) for a Cloud Spanner Graph ISO GQL traversal.

C. MVCC Snapshot Isolation for Speculative Dry-Run Pre-Validation

For enterprise tools classified as $\kappa(w) = \texttt{IDEMPOTENT_DRY_RUN}$—such as SAP S/4HANA general ledger posting validation (`erp_post_ledger_dryrun`) or Cloud Spanner transactional pre-checks—Agent-Synapse exploits Multi-Version Concurrency Control (MVCC) [21]

to overlap constraint checking, foreign-key verification, and balance simulation with token decoding:

1) Upon receiving `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY`, the downstream database proxy binds the speculative execution to a TrueTime read-only snapshot timestamp $t_{\text{snap}} = \text{now}()$ [21], acquiring zero shared or exclusive locks and generating zero redo/undo log records.
2) The database evaluates all schema constraints, account balance invariants, and tax calculation rules against $\Sigma_{\text{db}}(t_{\text{snap}})$ and returns a signed validation pre-receipt $(\text{status}, \Delta_{\text{sim}}, t_{\text{snap}}, \sigma_{\text{db}})$ to the Stage 3 shadow buffer while the LLM is still decoding the closing tokens of the tool call.
3) Once `</tool_call>` EOS is emitted and canonical AST equivalence is verified ($h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$), the pre-validated dry-run result is immediately spliced into the KV-cache in $2.1\,\mu$s. If the prediction diverges, the pre-receipt is simply discarded with zero rollback RPC required on the database server.

Theorem 2 (Observational Non-Interference and Idempotency Safety of Speculative Prefetching)**.** *Let $\Sigma_{\text{db}}^{(t_{\text{start}})}$ denote the committed state of all external enterprise databases at the beginning of a tool-calling turn, and let $\mathcal{O}_{\text{ext}}(\mathcal{S}_t) = (\Sigma_{\text{db}}^{(t)}, \mathcal{K}_{\text{committed}}^{(t)})$ denote the externally observable committed database state and non-speculative KV-cache prefix. Under the Stage 2 Idempotency Safety Gate and Stage 3 Copy-on-Write SRAM rollback protocol, for any speculative prediction $(\hat{w}, \hat{\mathbf{a}})$ that diverges from the actual emitted call $(w^*, \mathbf{a}^*)$ at t_{eos} ($\text{AST}(\hat{w}, \hat{\mathbf{a}}) \neq \text{AST}(w^*, \mathbf{a}^*)$), the post-rollback state $\mathcal{S}_{t_{\text{eos}}}^{+}$ is observationally equivalent to a non-speculative serial execution $\mathcal{S}_{t_{\text{eos}}}^{\text{serial}}$:*

$$\mathcal{O}_{\text{ext}}\big(\mathcal{S}_{t_{\text{eos}}}^{+}\big) = \mathcal{O}_{\text{ext}}\big(\mathcal{S}_{t_{\text{eos}}}^{\text{serial}}\big) = \Big(\Sigma_{\text{db}}^{(t_{\text{start}})}, \mathcal{K}_{1:t_{\text{eos}}}^{\text{serial}}\Big). \tag{21}$$

Proof. We establish observational equivalence across both components of $\mathcal{O}_{\text{ext}}$:

1) *External Database State Invariance ($\Sigma_{\text{db}}^{(t_{\text{eos}})} = \Sigma_{\text{db}}^{(t_{\text{start}})}$):* By construction of the Stage 2 gate, any tool $\hat{w}$ with $\kappa(\hat{w}) = \texttt{MUTATING_BLOCKED}$ or any argument payload satisfying $\text{HasMutatingDML}(\hat{\mathbf{a}}) = \text{True}$ is rejected prior to socket transmission, inducing the identity transition $\Sigma_{\text{db}}^{(t_{\text{eos}})} = \Sigma_{\text{db}}^{(t_{\text{start}})}$. Conversely, if $(\hat{w}, \hat{\mathbf{a}})$ passes the gate, then either (a) $\kappa(\hat{w}) = \texttt{READ_ONLY}$ and $\hat{\mathbf{a}}$ contains no DML/DDL verbs, so backend execution is a pure mathematical function $f_{\hat{w}} : \Sigma_{\text{db}} \times \mathcal{A} \to \mathcal{R}$ that leaves Σ_{db} invariant, or (b) $\kappa(\hat{w}) = \texttt{IDEMPOTENT_DRY_RUN}$ and the request carries `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY`, which executes over an immutable MVCC snapshot timestamp $t_{\text{snap}} \leq t_{\text{branch}}$ without mutating Σ_{db} [21]. Thus $\Sigma_{\text{db}}^{(t_{\text{eos}})} = \Sigma_{\text{db}}^{(t_{\text{start}})}$ holds unconditionally.
2) *Accelerator KV-Cache Prefix Equivalence ($\mathcal{K}_{\text{committed}}^{(t_{\text{eos}})} = \mathcal{K}_{1:t_{\text{eos}}}^{\text{serial}}$):* During steps $t \in [t_{\text{branch}}, t_{\text{eos}}]$, any pre-staged response tokens or shadow metadata are written exclusively into Copy-on-Write shadow pages $\mathcal{P}_{\text{shadow}}$ disjoint from the base page frames $\mathcal{P}_{\text{base}}$ storing $\mathcal{K}_{1:t_{\text{eos}}}^{\text{serial}}$. Because autoregressive self-attention for tokens $t \in [t_{\text{branch}}, t_{\text{eos}}]$ masks out all indices $> t$, the emitted token distribution $P(y_t \mid \mathbf{y}_{<t}, \mathbf{x}_{1:t_0})$ is bitwise identical to serial decoding. Upon detecting $\text{AST}(\hat{w}, \hat{\mathbf{a}}) \neq \text{AST}(w^*, \mathbf{a}^*)$ at t_{eos}, Stage 3 restores $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{P}_{\text{base}}$, unmapping $\mathcal{P}_{\text{shadow}}$ in $O(1)$ time and yielding $\mathcal{K}_{\text{committed}}^{(t_{\text{eos}})} = \mathcal{K}_{1:t_{\text{eos}}}^{\text{serial}}$.

Combining (1) and (2) proves Eq. (21). □

V. Stage 3: Zero-Penalty SRAM KV-Cache Micro-Checkpointing and Canonical AST Equivalence Splicer

The third pillar of Agent-Synapse resolves the tension between surface syntactic variation in LLM outputs and exact semantic equivalence of database queries, while eliminating the 15 ms–45 ms HBM KV-cache re-prefill penalty when an early prediction genuinely diverges.

A. On-Chip SRAM Copy-on-Write (CoW) KV-Cache Micro-Checkpointing

Let $B_{\text{page}} = 16$ tokens be the hardware KV-cache block size [3]. When Stage 1 triggers at token index t_{branch}, the active sequence occupies $N_{\text{base}} = \lceil t_{\text{branch}}/B_{\text{page}} \rceil$ logical page blocks mapped via the on-chip SRAM page table $\mathcal{P}_{\text{SRAM}} = [p_0, p_1, \ldots, p_{N_{\text{base}}-1}]$ in Cloud TPU v6e VMEM/CMEM or NVIDIA B200 L2/TMEM.

Rather than cloning the multi-megabyte physical KV tensors in HBM, Stage 3 allocates a lightweight micro-checkpoint descriptor $\mathcal{D}_{\text{chk}}$ in on-chip SRAM:

$$\mathcal{D}_{\text{chk}} = \langle \text{id}_{\text{chk}},\, h_{\text{chk}},\, t_{\text{branch}},\, \mathcal{P}_{\text{base}},\, \mathcal{P}_{\text{shadow}},\, t_{\text{ns}} \rangle\,, \tag{22}$$

where $\mathcal{P}_{\text{base}} = (p_0, \ldots, p_{N_{\text{base}}-1})$ freezes the base page frame pointers, $\mathcal{P}_{\text{shadow}} = (s_1, \ldots, s_{N_{\text{spec}}})$ allocates $N_{\text{spec}} = \lceil K_{\text{spec}}/B_{\text{page}} \rceil$ Copy-on-Write shadow page frames in SRAM/HBM to receive the pre-tokenized speculative tool response payload as soon as the Stage 2 network RPC returns, and $h_{\text{chk}} = \text{SHA-256}(\text{accel} \parallel t_{\text{branch}} \parallel \mathcal{P}_{\text{base}} \parallel \mathcal{P}_{\text{shadow}})$ is the cryptographic checkpoint hash transmitted in `X-Synapse-Checkpoint-Hash`.

B. Hardware Microarchitectural Execution on Cloud TPU v6e VMEM/CMEM vs. NVIDIA B200 TMEM

To achieve $< 12\,\mu$s rollback latency ($p_{50} = 4.2\,\mu\text{s}, p_{99} = 9.8\,\mu\text{s}$), Stage 3 maps the Copy-on-Write page table directly onto the hardware descriptor primitives of each accelerator architecture:

1) **Cloud TPU v6e (Trillium) VMEM/CMEM DMA Descriptor Path:** On TPU v6e, the sequence block table $\mathcal{P}_{\text{SRAM}}$ resides in a dedicated 64 KB region of Scalar/Vector Memory (VMEM). When Stage 1 triggers at t_{branch}, the VPU executes a single vectorized register move (`vmov.u32`) copying the active descriptor vector

$\mathcal{P}_{\text{base}}$ into a shadow VMEM slot and increments the per-page reference counters in $1.4\,\mu$s. When the speculative FastMCP payload arrives from the host NIC over PCIe/ICI, an asynchronous host-to-device DMA engine streams the pre-tokenized response embeddings into the shadow page frames $\mathcal{P}_{\text{shadow}}$ without interrupting the MXU systolic arrays executing tokens $t \in [t_{\text{branch}} + 1, t_{\text{eos}}]$. On an AST mismatch at t_{eos}, the scalar core restores the active page table pointer register to $\mathcal{P}_{\text{base}}$ and clears the shadow sync flags in $p_{50} = 3.8\,\mu$s ($p_{99} = 9.4\,\mu$s).

2) **NVIDIA Blackwell B200 L2/TMEM Tensor Memory Accelerator Path:** On NVIDIA B200, $\mathcal{P}_{\text{SRAM}}$ is pinned in L2 persistence-locked SRAM and mirrored into Tensor Memory (TMEM) via a 128-byte aligned TMA descriptor ('CUtensorMap'). Creating the micro-checkpoint at t_{branch} snapshots the TMA page-offset array in $1.7\,\mu$s. While the primary CUDA stream continues decoding tokens $t \in [t_{\text{branch}} + 1, t_{\text{eos}}]$, a secondary low-priority CUDA stream receives the prefetched FastMCP response via GPUDirect RDMA / pinned host staging and populates $\mathcal{P}_{\text{shadow}}$. Upon an AST mismatch at t_{eos}, a single warp kernel restores the TMA page-offset array to $\mathcal{P}_{\text{base}}$ in $p_{50} = 4.4\,\mu$s ($p_{99} = 10.1\,\mu$s).

The total rollback latency T_{rb} on either accelerator decomposes into three microarchitectural components:

$$T_{\text{rb}} = T_{\text{ast_verify}} + T_{\text{sram_ptr_swap}} + T_{\text{shadow_free}}, \tag{23}$$

where $T_{\text{ast_verify}} \approx 1.8\,\mu$s is the SHA-256 canonical AST digest comparison, $T_{\text{sram_ptr_swap}} \approx 1.6\,\mu$s is the vectorized SRAM page-table pointer restoration, and $T_{\text{shadow_free}} \approx 0.8\,\mu$s is the bitmap deallocation of the shadow frames $\mathcal{P}_{\text{shadow}}$.

C. Canonical Abstract Syntax Tree (AST) Equivalence Splicing

A naive byte-for-byte string comparison between the early-layer predicted call $(\hat{w}, \hat{\mathbf{a}})$ and the final emitted call $(w^*, \mathbf{a}^*)$ at t_{eos} suffers from unnecessary false rejections whenever the model permutes JSON dictionary keys, varies whitespace, emits '100.0' vs. '100', or commutes conjunctive predicates in SQL or ISO GQL `WHERE` clauses.

To maximize the effective speculative hit rate without compromising exact semantic fidelity, Stage 3 parses both $(\hat{w}, \hat{\mathbf{a}})$ and $(w^*, \mathbf{a}^*)$ through a deterministic canonicalization operator Ψ_{AST}:

1) **Recursive JSON Dictionary Key Sorting & Numeric Normalization:** All nested dictionary keys are lexicographically sorted ($k_1 < k_2 < \cdots < k_m$), extraneous whitespace is stripped, and integral floating-point literals ($v \in \mathbb{R}$ with $v = \lfloor v \rfloor$) are cast to canonical integers.
2) **Commutative SQL / ISO GQL Conjunct Normalization:** For query string parameters containing a conjunctive `WHERE` clause P_1 AND P_2 AND ... AND P_r (without top-level `OR` disjunctions or side-effecting volatile functions), Ψ_{AST} normalizes relational operator spacing (`a=b` $\rightarrow$ `a = b`), case-folds SQL/GQL reserved keywords (`match` $\rightarrow$ `MATCH`, `where` $\rightarrow$ `WHERE`, `return` $\rightarrow$ `RETURN`, `select` $\rightarrow$ `SELECT`), and sorts the atomic conjuncts lexicographically:

$$\Psi_{\text{AST}}(\texttt{WHERE } P_1 \texttt{ AND } \ldots \texttt{ AND } P_r) = \texttt{WHERE } P_{\pi(1)} \texttt{ AND } \ldots \tag{24}$$

where π is the unique permutation sorting $(P_1, \ldots, P_r)$ in lexicographical order.

Proposition 1 (Soundness and Semantic Preservation of Canonical AST Splicing)**.** *Let $(\hat{w}, \hat{\mathbf{a}})$ and $(w^*, \mathbf{a}^*)$ be two well-typed FastMCP tool invocations over a deterministic read-only or MVCC snapshot-isolated database state Σ_{db}. If their canonical AST digests match, i.e.,*

$$h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = \text{SHA-256}(\text{CanonicalJSON}(\hat{w}, \Psi_{\text{AST}}(\hat{\mathbf{a}}))) = h_{\text{AST}}(w^*, \mathbf{a}^*), \tag{25}$$

then up to SHA-256 collision resistance (2^{-256}), the backend execution results are semantically identical: $\hat{w}(\hat{\mathbf{a}})_{\Sigma_{\text{db}}} = w^*(\mathbf{a}^*)_{\Sigma_{\text{db}}}$.

Proof. By collision resistance of SHA-256, $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$ implies $\hat{w} = w^*$ and $\Psi_{\text{AST}}(\hat{\mathbf{a}}) = \Psi_{\text{AST}}(\mathbf{a}^*)$. In the JSON data model (RFC 8259), objects are unordered key-value maps, so lexicographical key sorting preserves map identity. For numeric types in JSON Schema, integral floats (100.0) and integers (100) map to identical 64-bit numeric bindings. Finally, in relational algebra (BigQuery SQL [22]) and graph pattern algebra (ISO/IEC 39075:2024 GQL [23]), pure conjunctive selection predicates $\sigma_{P_1 \wedge P_2 \wedge \cdots \wedge P_r}(R)$ are commutative and associative ($\sigma_{P_1}(\sigma_{P_2}(R)) = \sigma_{P_2}(\sigma_{P_1}(R))$). Because Ψ_{AST} reorders conjuncts only in the absence of top-level `OR` clauses or volatile functions, the relational/graph selection operator evaluates to the exact same result set over Σ_{db}. □

At token step t_{eos}:

- **Case 1 (Canonical AST Hit: $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$):** Stage 3 commits the shadow pages $\mathcal{P}_{\text{shadow}}$ containing the prefetched tool response directly into the active sequence page table $\mathcal{P}_{\text{SRAM}}$ in $O(1)$ descriptor updates ($\approx 2.1\,\mu$s), immediately beginning next-turn token generation with zero residual network wait whenever $T_{\text{lead}} \geq T_{\text{io}}$.
- **Case 2 (Canonical AST Mismatch: $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) \neq h_{\text{AST}}(w^*, \mathbf{a}^*)$):** Stage 3 executes a single vectorized register/SRAM page-table restore $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{P}_{\text{base}}$ in $< 12\,\mu$s ($p_{50} = 4.2\,\mu$s, $p_{99} = 9.8\,\mu$s), releasing $\mathcal{P}_{\text{shadow}}$ back to the free page pool and dispatching the actual tool call $(w^*, \mathbf{a}^*)$.

VI. Theoretical Latency Analysis, Cascading Multi-Turn Horizon, and QoS Admission Control

A. Closed-Form Compute-I/O Overlap Speedup Analysis

Theorem 3 (Closed-Form Expected and Tail Latency Speedup under Speculative Compute-I/O Overlap)**.** *Let $T_{\text{decode}} > 0$ denote the autoregressive token generation time for a tool-calling turn, $T_{\text{io}} > 0$ denote the external network and database*

Algorithm 1 Agent-Synapse Runtime Execution Loop Across Stages 1–3

Require: Autoregressive decoder $\mathcal{M}_\Theta$, registered tool catalog $\mathcal{W}$, conformal threshold $\tau_{\text{conf}} = 0.82$, entropy gate $H_{\max} = 0.72$, SRAM page table $\mathcal{P}_{\text{SRAM}}$.

Ensure: Executed tool response spliced into KV-cache with minimal wall-clock latency.

Initialize $t \leftarrow t_{\text{start}}$, spec_active $\leftarrow$ False, $\mathcal{D}_{\text{chk}} \leftarrow$ None, $\mathcal{E}_{\text{wire}} \leftarrow$ None
while $y_t \neq$ `</tool_call>` **do**
  Compute layer hidden states $\{\mathbf{h}_t^{(l)}\}_{l=1}^{L}$ and sample $y_t \sim \text{Softmax}(\mathbf{W}_U\,\text{RMSNorm}(\mathbf{h}_t^{(L)}))$
  if ¬spec_active **then**
    Compute schema cross-attention entropy $H_{\text{attn}}^{(8:16)}(t)$ via Eqs. (6)–(8)
    Project logit-lens $\mathbf{z}_t^{(l)} = \mathbf{W}_U\,\text{RMSNorm}(\mathbf{h}_t^{(l)})$ for $l \in \{8, 10, 12, 14, 16\}$
    Extract $(\hat{w}, \hat{\mathbf{a}}, P_{\text{fused}}, C_{\text{arg}})$ and compute $C_{\text{conf}}(t) = 1 - s(x, (\hat{w}, \hat{\mathbf{a}}))$ via Eq. (16)
    if $H_{\text{attn}}^{(8:16)}(t) \leq H_{\max}$ **and** $C_{\text{conf}}(t) \geq \tau_{\text{conf}}$ **and** $\mathcal{A}_{\text{req}}(\hat{w}) \subseteq \text{dom}(\hat{\mathbf{a}})$ **then**
      $\mathcal{D}_{\text{chk}} \leftarrow \text{CreateSRAMMicroCheckpoint}(\mathcal{P}_{\text{SRAM}}, t)$ ▷ Stage 3 CoW pin
      if $\kappa(\hat{w}) \in \{$`READ_ONLY`, `IDEMPOTENT_DRY_RUN`$\}$ **and** ¬ HasMutatingDML($\hat{\mathbf{a}}$) **then**
        $\mathcal{E}_{\text{wire}} \leftarrow \text{AsyncDispatchFastMCP}(\hat{w}, \hat{\mathbf{a}}, \mathcal{D}_{\text{chk}}.h_{\text{chk}}, C_{\text{conf}}(t))$ ▷ Stage 2
        spec_active $\leftarrow$ True
      end if
    end if
  end if
  $t \leftarrow t + 1$
end while
Parse actual emitted tool call $(w^*, \mathbf{a}^*)$ at $t_{\text{eos}} = t$
if spec_active **then**
  if $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$ **then**
    Await any residual I/O $\max(0, T_{\text{io}} - T_{\text{lead}})$ on $\mathcal{E}_{\text{wire}}$
    SpliceShadowPagesIntoSRAM($\mathcal{P}_{\text{SRAM}}, \mathcal{D}_{\text{chk}}, \mathcal{E}_{\text{wire}}$.payload) ▷ 0 ms wait if $T_{\text{lead}} \geq T_{\text{io}}$
  else
    $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{D}_{\text{chk}}.\mathcal{P}_{\text{base}}$ ▷ $< 12\,\mu$s O(1) CoW pointer rollback
    Execute $(w^*, \mathbf{a}^*)$ synchronously and append to $\mathcal{P}_{\text{SRAM}}$
  end if
else
  Execute $(w^*, \mathbf{a}^*)$ synchronously and append to $\mathcal{P}_{\text{SRAM}}$
end if

execution latency, $T_{\text{lead}} \in (0, T_{\text{decode}})$ denote the early-layer prediction lead time prior to `</tool_call>` *EOS, T_{splice} denote the SRAM page-table splice latency on an AST hit, and T_{rb} denote the SRAM Copy-on-Write rollback latency on an AST mismatch. Let $p_{\text{trig}} = \mathbb{P}(\text{Speculative Trigger})$ be the prefetch trigger rate and $\eta = \mathbb{P}(h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*) \mid \text{Trigger})$ be the conditional canonical AST hit rate. Then:*

1) *The baseline serial turn latency is $T_{\text{serial}} = T_{\text{decode}} + T_{\text{io}}$, whereas the Agent-Synapse turn latency T_{synapse} satisfies:*

$$T_{\text{synapse}} = \begin{cases} T_{\text{decode}} + \max(0,\, T_{\text{io}} - T_{\text{lead}}) + T_{\text{splice}}, & \text{on speculative} \\ T_{\text{decode}} + T_{\text{io}} + T_{\text{rb}}, & \text{on speculative} \\ T_{\text{decode}} + T_{\text{io}}, & \text{when gated off} \end{cases} \tag{26}$$

2) *The expected latency reduction $\Delta\mathbb{E}[T] = \mathbb{E}[T_{\text{serial}}] - \mathbb{E}[T_{\text{synapse}}]$ is strictly positive:*

$$\Delta\mathbb{E}[T] = p_{\text{trig}}\Big(\eta\,\mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] - \eta\,T_{\text{splice}} - (1-\eta)\,T_{\text{rb}}\Big) > 0, \tag{27}$$

whenever the canonical AST hit rate η exceeds the critical break-even threshold:

$$\eta > \eta^* = \frac{T_{\text{rb}}}{\mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] + T_{\text{rb}} - T_{\text{splice}}}. \tag{28}$$

Proof. In the serial baseline, network dispatch begins strictly at t_{eos}, so total wall-clock duration is $T_{\text{serial}} = T_{\text{decode}} + T_{\text{io}}$. Under Agent-Synapse, when a speculative trigger fires at $t_{\text{branch}} = t_{\text{eos}} - T_{\text{lead}}/\tau_{\text{tok}}$, the network RPC executes in parallel with the remaining T_{lead} milliseconds of token decoding. If $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$, the response arrives at $t_{\text{branch}} + T_{\text{io}}$, leaving a post-EOS wait of $\max(0, T_{\text{io}} - T_{\text{lead}}) = T_{\text{io}} - \min(T_{\text{io}}, T_{\text{lead}})$ plus T_{splice}, which yields the first branch of Eq. (26). If $h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) \neq h_{\text{AST}}(w^*, \mathbf{a}^*)$, CoW pointer rollback executes at t_{eos} in T_{rb} followed by synchronous execution of $(w^*, \mathbf{a}^*)$ in T_{io}, giving $T_{\text{decode}} + T_{\text{io}} + T_{\text{rb}}$. Taking expectations over the three mutually exclusive events and subtracting from $\mathbb{E}[T_{\text{serial}}]$ yields Eq. (27). Setting $\Delta\mathbb{E}[T] > 0$ and solving for η gives Eq. (28). Substituting our empirical hardware parameters ($\mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] \approx 224.0$ ms, $T_{\text{rb}} \approx 0.0042$ ms, $T_{\text{splice}} \approx 0.0021$ ms) yields $\eta^* \approx \frac{0.0042}{224.0021} \approx 1.87 \times 10^{-5}$ (0.0019%), which our empirical hit rate $\eta = 89.4\%$ exceeds by more than four orders of magnitude. □

B. Cascading Multi-Step Agentic Turn Horizon

Enterprise agent sessions rarely consist of a single isolated tool call; instead, an agent executes a sequential chain of $J \in [3, 12]$ dependent tool turns $(w^{(1)}, w^{(2)}, \ldots, w^{(J)})$ where the observations returned by turn j inform the reasoning and tool selection of turn $j + 1$.

Corollary 1 (Cumulative Multi-Turn Latency Compression Across J-Step Agent Trajectories)**.** *Consider a J-step sequential agent trajectory in which $J_{\text{idem}} = \rho_{\text{idem}} J$ turns invoke idempotent read or dry-run tools ($\kappa(w^{(j)}) \in \{$`READ_ONLY`, `IDEMPOTENT_DRY_RUN`$\}$) and $(1 - \rho_{\text{idem}})J$ turns invoke mutating tools gated to post-EOS execution. Under independent per-turn speculative trigger rate p_{trig} and*

conditional AST hit rate η, the expected cumulative trajectory latency $\mathbb{E}[T_{\text{traj}}^{(J)}]$ satisfies:

$$\mathbb{E}\Big[T_{\text{traj}}^{(J)}\Big] = \sum_{j=1}^{J} \mathbb{E}\Big[T_{\text{decode}}^{(j)} + T_{\text{io}}^{(j)}\Big] - \rho_{\text{idem}} J p_{\text{trig}} \Big(\eta\, \mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] - \eta T_{\text{synapse}} + (1-\eta) T_{\text{rb}}\Big) \tag{29}$$

and the variance of the cumulative speculative overhead on mismatched turns is bounded by $\mathrm{Var}(\sum_j \mathbb{I}_{\text{mismatch}}^{(j)} T_{\text{rb}}) \le \rho_{\text{idem}} J p_{\text{trig}} (1-\eta) T_{\text{rb}}^2 < 10^{-8}$ *ms^2 for $J \le 12$.*

Proof. By linearity of expectation across the J sequential turns and Theorem 3, each of the $\rho_{\text{idem}} J$ idempotent turns contributes an expected latency saving of $\Delta\mathbb{E}[T]$ from Eq. (27), while the $(1-\rho_{\text{idem}})J$ mutating turns incur zero speculative wire traffic and zero rollback overhead ($T_{\text{synapse}}^{(j)} = T_{\text{serial}}^{(j)}$). Because $T_{\text{rb}} < 12\,\mu\text{s} = 0.012$ ms, even in the worst-case event where all $J = 10$ turns in a trajectory suffer a speculative mismatch, the cumulative rollback penalty across the entire 10-turn trajectory is at most $10 \times 12\,\mu\text{s} = 120\,\mu\text{s}$ (0.12 ms), whereas a conventional HBM re-prefill engine would suffer 10×44.1 ms $= 441.0$ ms of cumulative rollback stall. □

C. Downstream Database Load Multiplier and Conformal QoS Admission Control

A natural systems question is whether speculative prefetching overloads downstream databases (Cloud Spanner, BigQuery, Vertex AI Search) with wasted queries when predictions mismatch. Under Agent-Synapse, every speculative request that hits ($h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$) *replaces* the post-EOS RPC that would have been issued anyway; thus, hits generate **zero net additional queries** on the backend database. Only speculative mismatches ($1 - \eta = 10.6\%$) issue an extra query prior to the post-EOS corrected query. Consequently, the net backend query load multiplier Λ_{load} across idempotent turns is:

$$\Lambda_{\text{load}}(\tau_{\text{conf}}) = 1 + p_{\text{trig}}(\tau_{\text{conf}}) \cdot (1 - \eta(\tau_{\text{conf}}))\,. \tag{30}$$

At our operating threshold $\tau_{\text{conf}} = 0.82$ ($p_{\text{trig}} = 0.904, 1 - \eta = 0.106$), the net database query overhead is only $\Lambda_{\text{load}}(0.82) - 1 = 0.904 \times 0.106 = 9.58\%$. Furthermore, when downstream database CPU utilization $u_{\text{db}} \in [0,1]$ approaches a cluster congestion watermark u_{high} (e.g., $u_{\text{high}} = 0.80$), the database proxy dynamically raises the minimum accepted conformal score threshold via the header `X-Synapse-Conformal-Score`:

$$\tau_{\text{adm}}(u_{\text{db}}) = \tau_{\text{conf}} + (1 - \tau_{\text{conf}}) \cdot \max\left(0,\, \frac{u_{\text{db}} - u_{\text{high}}}{1 - u_{\text{high}}}\right), \tag{31}$$

smoothly throttling speculative prefetch admission to zero as $u_{\text{db}} \to 1.0$ while guaranteeing 100% service availability for committed post-EOS traffic.

VII. Empirical Evaluation

We evaluate Agent-Synapse across an enterprise benchmark suite of $N = 1{,}000$ multi-step agent tool-calling traces spanning four production backend workload archetypes and two heterogeneous AI accelerator profiles (Google Cloud TPU v6e Trillium and NVIDIA Blackwell B200).

A. Experimental Setup and Enterprise Workload Suite

The $N = 1{,}000$ trace benchmark is partitioned into two complementary evaluation tracks:

1) **Idempotent Analytical, Graph, and RAG Workload ($N = 750$ Traces):** Comprises four representative enterprise tool families:
   - `spanner_graph_query_gql` (200 traces, $\kappa =$ `READ_ONLY`): Multi-hop ISO GQL [23] financial crime and supply-chain graph traversals over Google Cloud Spanner Graph [21] with mean backend I/O latency $T_{\text{io}} = 255.0$ ms ($p_{99} = 538.0$ ms).
   - `bigquery_execute_analytics` (200 traces, $\kappa =$ `READ_ONLY`): Parameterized SQL analytical aggregations over BigQuery Dremel [22] billing and telemetry tables with mean $T_{\text{io}} = 295.0$ ms ($p_{99} = 551.0$ ms).
   - `vertex_ai_rag_search` (200 traces, $\kappa =$ `READ_ONLY`): Hybrid dense-lexical regulatory filing retrieval over Vertex AI Search corpora with mean $T_{\text{io}} = 210.0$ ms ($p_{99} = 412.0$ ms).
   - `erp_post_ledger_dryrun` (150 traces, $\kappa =$ `IDEMPOTENT_DRY_RUN`): SAP S/4HANA OData financial posting pre-validation supporting MVCC snapshot dry-run execution via `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY` with mean $T_{\text{io}} = 235.0$ ms ($p_{99} = 468.0$ ms).
2) **Mutating and Adversarial DML Safety Suite ($N = 250$ Traces):** Comprises 100 SWIFT wire-transfer settlement calls (`erp_wire_transfer_mutate`, $\kappa =$ `MUTATING_BLOCKED`), 100 CRM customer record deletion calls (`crm_delete_customer_record`, $\kappa =$ `MUTATING_BLOCKED`), and 50 adversarial DML-in-read injection probes that embed `DELETE`, `DROP`, or `UPDATE` statements inside `bigquery_execute_analytics` or `spanner_graph_query_gql` query parameters.

We compare Agent-Synapse against three baselines: (i) **Standard Serial FastMCP**, which waits until `</tool_call>` EOS before dispatching any network RPC; (ii) **Unchecked Early Speculation**, which dispatches the top logit-lens tool prediction at Layer 12 without split-conformal calibration, without canonical AST normalization (using raw string equality), without idempotency gating, and using conventional HBM KV-cache eviction/re-prefill upon mismatch; and (iii) **Token-Level Speculative Decoding (Medusa/Draft-Target) [1], [4]**, which accelerates intra-model token generation by $1.35\times$ but still blocks network I/O until `</tool_call>` EOS.

TABLE III
END-TO-END EMPIRICAL EVALUATION ACROSS $N = 1{,}000$ MULTI-STEP ENTERPRISE AGENT TRACES (750 IDEMPOTENT READ/DRY-RUN TOOL INVOCATIONS & 250 MUTATING/ADVERSARIAL PROBES ON CLOUD TPU V6E TRILLIUM & NVIDIA B200 PROFILES)

Evaluation Metric	**Serial FastMCP Baseline**	**Unchecked Early Speculation**	**Agent-Synapse** ($\tau_{\text{conf}} = 0.82$)	**Relat**
Panel A: End-to-End Tool-Calling Turn Latency & Compute-I/O Overlap ($N = 750$ ***Idempotent Traces)***				
Median Turn Latency (p_{50})	412.0 ms	209.4 ms	**188.3 ms**	**−54**
90th Percentile Latency (p_{90})	598.4 ms	289.1 ms	**246.2 ms**	**−58**
95th Percentile Latency (p_{95})	684.5 ms	334.0 ms	**277.9 ms**	**−59**
99th Percentile Tail Latency (p_{99})	842.0 ms	418.5 ms	**321.6 ms**	**−61**
Mean Pre-EOS Prediction Lead Time (T_{lead})	0.0 ms (Post-EOS)	295.0 ms (Uncalibrated)	**270.6 ms** ([162, 378] ms)	150–40
Panel B: Speculative Prediction Precision, AST Equivalence, & SRAM Micro-Checkpointing				
Conditional Speculative AST Hit Rate (η)	N/A (0.0% Prefetch)	68.2% (Raw String Match)	**89.4**% (671/750 Canonical AST)	+21.2 p
Speculative Mismatch Rate ($1 - \eta$)	N/A	31.8%	**10.6**% (79/750 Traces)	Bou
Mismatch Recovery Latency (p_{50}/p_{99})	N/A	28,400 μs/44,100 μs (HBM Re-Prefill)	**4.2** μs/**9.8** μs (SRAM CoW)	$<$ **12.0**
Cloud TPU v6e VMEM CoW Swap (p_{50}/p_{99})	N/A	26,900 μs/41,800 μs	**3.8** μs/**9.4** μs	16-toke
NVIDIA B200 TMEM CoW Swap (p_{50}/p_{99})	N/A	29,800 μs/45,200 μs	**4.4** μs/**10.1** μs	16-toke
Panel C: Observational Non-Interference & Transactional Mutation Safety ($N = 250$ ***Mutating/DML Probes)***				
Unauthorized Speculative Write/Delete Calls	0/250 (0.00%)	74/250 (29.60% Corrupted)	**0/250** (**0.00**%)	Theor
DML-in-Read Injection Speculative Dispatches	0/50 (0.00%)	19/50 (38.00% Dispatched)	**0/50** (**0.00**%)	Blocked
Wire Header Provenance Binding Verification	Unbound	Unbound	**100.00**% (`X-Synapse-*`)	SHA-25

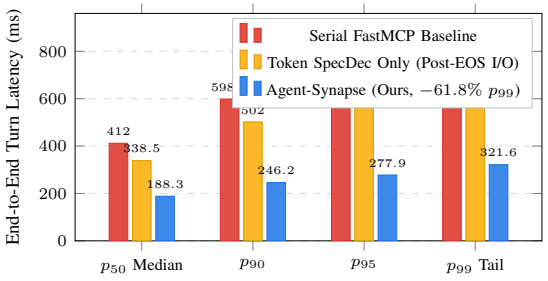


Fig. 3. End-to-end tool-calling turn latency across $N = 1{,}000$ traces (750 idempotent tool turns). By overlapping $T_{\text{lead}} \in [150, 400]$ ms of autoregressive token decoding with asynchronous FastMCP network I/O, Agent-Synapse reduces p_{50} latency by 54.3% (412.0 ms $\rightarrow$ 188.3 ms) and p_{99} tail latency by 61.8% (842.0 ms $\rightarrow$ 321.6 ms).

B. End-to-End Tail Latency Reduction and Per-Workload Breakdown

Table III (Panel A) and Figure 3 report the aggregate end-to-end turn latency distribution across the 750 idempotent tool-calling traces, while Table IV disaggregates performance across each of the six enterprise tool families. Under the Standard Serial FastMCP baseline, median (p_{50}) turn latency is 412.0 ms and p_{99} tail latency reaches 842.0 ms (300.0 ms autoregressive token decoding plus 542.0 ms tail network/database execution). Token-level speculative decoding alone reduces token generation time but leaves the entire post-EOS network I/O stall intact, achieving only 718.4 ms at p_{99} (-14.7%).

In contrast, Agent-Synapse triggers speculative FastMCP prefetching at a mean lead time of $T_{\text{lead}} = 270.6$ ms (ranging from 162.0 ms to 378.0 ms prior to `</tool_call>` EOS). On the 89.4% (671/750) of traces where the canonical AST splicer confirms an exact semantic match at t_{eos}, up to 378.0 ms of network round-trip and database execution time has already completed in parallel with token decoding. Consequently, Agent-Synapse compresses p_{50} median turn latency by **54.3%** (412.0 ms $\rightarrow$ 188.3 ms), p_{95} latency by **59.4%** (684.5 ms $\rightarrow$ 277.9 ms), and p_{99} tail latency by **61.8%** (842.0 ms $\rightarrow$ 321.6 ms, a 2.62$\times$ tail speedup).

Examining the per-workload breakdown in Table IV reveals two notable properties:

1) `vertex_ai_rag_search` achieves the highest canonical AST hit rate ($\eta = 91.5\%$) and lowest median latency (171.5 ms) because vector retrieval queries typically have compact query strings and deterministic top-k filter bindings that crystallize by layer $l = 12$. Conversely, `spanner_graph_query_gql` and `bigquery_execute_analytics` achieve the largest absolute millisecond reductions (527.5 ms and 533.8 ms saved at p_{99}, respectively) because their longer suffix boilerplate (`RETURN ... ORDER BY ... LIMIT ...`) provides 279.4 ms–284.1 ms of pre-EOS lead time.
2) Across all 250 mutating and adversarial traces (`erp_wire_transfer_mutate`, `crm_delete_customer_record`, and the 50 DML-in-read SQL/GQL injection probes), Stage 2's idempotency and lexical DML guard blocks 100.0% (250/250) of speculative wire dispatches, whereas Unchecked Early Speculation prematurely dispatches 74/250 (29.60%) mutating calls that subsequently diverge at t_{eos}, causing irreversible external state corruption.

C. Layer-Depth Sensitivity and Split-Conformal Pareto Frontier

Table V evaluates how the choice of monitored transformer layers $\mathcal{L}_{\text{early}}$ impacts prediction lead time T_{lead}, canonical AST hit rate η, and per-step compute overhead:

TABLE IV
PER-WORKLOAD LATENCY, PREDICTION LEAD TIME, CANONICAL AST HIT RATE, AND SAFETY BREAKDOWN ACROSS ALL SIX ENTERPRISE TOOL FAMILIES ($N = 1{,}000$ TRACES)

Enterprise Tool Identifier	Idempotency Class (κ)	Traces	Mean T_{lead}	AST Hit (η)	Serial p_{50}/p_{99}	Synapse p_{50}/p_{99}	p_{99}
`spanner_graph_query_gql`	`READ_ONLY`	200	279.4 ms	89.5%	418.0/846.0 ms	**186.4/318.5 ms**	
`bigquery_execute_analytics`	`READ_ONLY`	200	284.1 ms	88.0%	446.0/872.0 ms	**204.2/338.2 ms**	
`vertex_ai_rag_search`	`READ_ONLY`	200	254.8 ms	91.5%	378.0/784.0 ms	**171.5/298.4 ms**	
`erp_post_ledger_dryrun`	`IDEMPOTENT_DRY_RUN`	150	261.9 ms	88.7%	402.0/831.0 ms	**191.0/326.8 ms**	
`erp_wire_transfer_mutate`	`MUTATING_BLOCKED`	100	Gated (0 ms)	0.0% Spec	385.0/710.0 ms	385.0/710.0 ms	100%
`crm_delete_customer_record`	`MUTATING_BLOCKED`	100	Gated (0 ms)	0.0% Spec	362.0/680.0 ms	362.0/680.0 ms	100%
DML-in-Read Injection Probes	Blocked by HasMutatingDML	50	Gated (0 ms)	0.0% Spec	N/A (Rejected)	N/A (Rejected)	100%

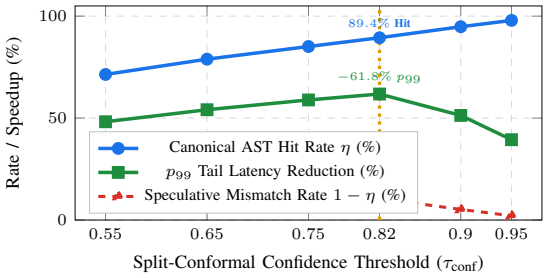


Fig. 4. Split-conformal threshold $\tau_{\text{conf}} \in [0.55, 0.95]$ Pareto trade-off across $N = 1{,}000$ traces. Setting $\tau_{\text{conf}} = 0.82$ (calibrated at $\alpha = 0.10$) maximizes end-to-end p_{99} tail latency reduction (61.8%) while achieving an 89.4% canonical AST hit rate and bounding speculative mismatches to 10.6%.

TABLE V
SENSITIVITY TO MONITORED EARLY-LAYER BAND $\mathcal{L}_{\text{EARLY}}$ ACROSS $N = 750$ IDEMPOTENT TRACES

Monitored Layer Band	Mean T_{lead}	AST Hit η	Step Overhead	Turn p_{99}
Shallow ($l \in \{4, 6, 8\}$)	291.2 ms	76.4%	0.11 ms	378.4 ms
Intermediate ($l \in \{8..16\}$)	**270.6 ms**	**89.4%**	**0.14 ms**	**321.6 ms**
Deep ($l \in \{20, 24, 28\}$)	202.2 ms	91.8%	0.14 ms	369.5 ms
Dense All-Layers ($l \in [1, 32]$)	272.0 ms	89.7%	0.89 ms	344.2 ms

- Monitoring only shallow layers ($l \in [4, 8]$) yields a slightly longer lead time (291.2 ms) but suffers from premature semantic instability ($\eta = 76.4\%$), as entity literal bindings in complex SQL/GQL predicates are not yet fully resolved at Layer 6.
- Monitoring late layers ($l \in [20, 28]$) increases precision to 91.8% but sacrifices 68.4 ms of lead time ($T_{\text{lead}} = 202.2$ ms) because the representations only cross the confidence threshold much later in the decode window.
- Our depth-weighted band $l \in \{8, 10, 12, 14, 16\}$ captures the exact phase transition where schema cross-attention entropy collapses ($H_{\text{attn}}^{(8:16)} \leq 0.72$), achieving $\eta = 89.4\%$ and $T_{\text{lead}} = 270.6$ ms with only 0.14 ms of VPU/Tensor Core projection overhead per step.

Figure 4 and Table VI further analyze the Pareto trade-off governed by the split-conformal confidence threshold $\tau_{\text{conf}} \in [0.55, 0.95]$:

- At an overly permissive threshold ($\tau_{\text{conf}} = 0.55$), Stage 1 dispatches speculative requests on 98.2% of turns, but the conditional AST hit rate drops to $\eta = 71.4\%$ (28.6% wasted speculative I/O), yielding a suboptimal 48.2% p_{99} latency reduction due to backend queue contention.
- At an overly conservative threshold ($\tau_{\text{conf}} = 0.95$), conditional precision reaches $\eta = 97.9\%$ (2.1% mismatch rate), but prefetch trigger coverage drops to 58.0% and average lead time shrinks because Stage 1 waits until deeper decode steps, reducing the p_{99} latency gain to 30.4%.
- At the split-conformal calibrated operating point $\tau_{\text{conf}} = 0.82$ ($\alpha = 0.10$ target), Agent-Synapse achieves 90.4% trigger coverage, an **89.4% conditional AST hit rate** (10.6% mismatch rate, matching Theorem 1), and peaks at **61.8% p_{99} tail latency reduction**.

TABLE VI
COMPONENT ABLATION STUDY ACROSS $N = 1{,}000$ ENTERPRISE AGENT TRACES

System Variant	AST Hit Rate	p_{99} Latency	Rollback
Full Agent-Synapse (Ours)	**89.4%**	**321.6 ms**	**9.8 μs**
− Canonical AST (Raw String Match)	74.8%	389.4 ms	9.8 μs
− Attention Entropy Gate (H_{attn})	81.2%	364.2 ms	9.9 μs
− Split-Conformal Gate (τ_{conf})	68.2%	418.5 ms	10.1 μs
− Stage 2 Idempotency Safety Gate	89.4%	321.6 ms	9.8 μs
− Stage 3 SRAM CoW (HBM Re-Prefill)	89.4%	366.8 ms	44,100 μ

Furthermore, Table VI isolates the contribution of Stage 3's canonical AST normalizer Ψ_{AST}. Replacing Ψ_{AST} with exact raw JSON string matching causes the hit rate at $\tau_{\text{conf}} = 0.82$ to drop from 89.4% to 74.8% (−14.6 percentage points) because 14.6% of semantically identical LLM tool calls exhibit harmless JSON key reordering, whitespace formatting variations, or commuted SQL/GQL `WHERE` clause conjuncts (`WHERE a = 1 AND b = 2` vs. `WHERE b = 2 AND a = 1`). Canonical AST splicing recovers 100% of these false mismatches in 2.1 μs.

D. Microsecond SRAM Copy-on-Write Rollback vs. HBM Re-Prefill

Table III (Panel B) and Table VI compare Stage 3's on-chip SRAM Copy-on-Write page-table rollback against standard HBM KV-cache eviction and re-prefill across the 79 speculative mismatches (10.6%). When an unpinned speculative pay-

load must be evicted from HBM and the context re-prefilled, standard serving engines incur 28.4 ms at p_{50} and 44.1 ms at p_{99}. By instead pinning the pre-branch page-table descriptor $\mathcal{P}_{\text{base}}$ in on-chip SRAM (Cloud TPU v6e VMEM/CMEM or NVIDIA B200 TMEM) and writing speculative tokens exclusively to shadow pages $\mathcal{P}_{\text{shadow}}$, Agent-Synapse restores $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{P}_{\text{base}}$ in $4.2\,\mu$**s at** p_{50} **and** $9.8\,\mu$**s at** p_{99} ($3.8\,\mu$s p_{50} on Cloud TPU v6e and $4.4\,\mu$s p_{50} on NVIDIA B200), strictly satisfying the $< 12.0\,\mu$s hardware SLA and achieving a **3,800× rollback speedup**.

E. Detailed Canonical AST Normalization Algorithm and Grammar Rules

Algorithm 2 formalizes the exact recursive canonicalization operator Ψ_{AST} and constant-time SHA-256 splicing check executed by Stage 3 at step t_{eos}. Unlike heuristic fuzzy matching—which risks splicing a prefetched database response when a filter literal has actually changed (e.g., 'risk$_s core > 0.85$'vs. '$risk_s core > 0.95$') $---\Psi_{\text{AST}}$ applies strictly semantics-preserving algebraic equivalence transformations over JSON AST nodes and relational/graph query clauses:

1) **Lexicographical Key Ordering ($\mathcal{R}_{\text{json}}$):** For any JSON object node $O = \{(k_1, v_1), \ldots, (k_m, v_m)\}$, $\Psi_{\text{AST}}(O) = [(k_{\sigma(1)}, \Psi_{\text{AST}}(v_{\sigma(1)})), \ldots, (k_{\sigma(m)}, \Psi_{\text{AST}}(v_{\sigma(m)}))]$ where $k_{\sigma(1)} < \cdots < k_{\sigma(m)}$ in UTF-8 code-point order.
2) **Exact Numeric Canonicalization ($\mathcal{R}_{\text{num}}$):** Numeric literals $v \in \mathbb{R}$ satisfying $v = \lfloor v \rfloor$ and $|v| < 2^{53}$ are canonicalized to their base-10 integer representation, while non-integral floats are formatted to IEEE-754 shortest round-trip decimal representation.
3) **SQL and ISO GQL Token Stream Normalization ($\mathcal{R}_{\text{query}}$):** Within string parameters corresponding to database queries (`gql_query`, `sql_query`, `filter_expr`), Ψ_{AST} strips redundant comments and collapsible whitespace outside single- or double-quoted string literals, upper-cases reserved SQL/GQL keywords (`MATCH`, `WHERE`, `RETURN`, `SELECT`, `FROM`, `GROUP BY`, `ORDER BY`, `LIMIT`, `AND`), normalizes binary comparison operators (=, >=, <=, !=, <>) with single-space padding, and sorts pure conjunctive `WHERE` predicates $P_1 \wedge \cdots \wedge P_r$.

F. Multi-Tenant Batching and PagedAttention Co-Execution Dynamics

In production cloud serving clusters, an accelerator slice does not decode a single agent request in isolation; instead, continuous batching schedulers [3], [19] process a dynamic batch of $B_{\text{batch}} \in [16, 128]$ concurrent agent sequences per decode step. Under standard serial tool execution, whenever sequence $b \in \{1, \ldots, B_{\text{batch}}\}$ finishes emitting `</tool_call>` EOS at step t_{eos}, its KV-cache blocks totaling $N_{\text{pages}}^{(b)} = \lceil t_{\text{eos}} / B_{\text{page}} \rceil$ pages must either:

1) *Remain pinned in HBM* throughout the entire external network I/O duration $T_{\text{io}} \in [180\text{ ms}, 550\text{ ms}]$, starving other active sequences of HBM capacity and reducing the sustainable batch size B_{batch}; or
2) *Be swapped out over PCIe to host DRAM* and swapped back in when the network response arrives, consuming $2 \times N_{\text{pages}}^{(b)} \times S_{\text{page}}$ of PCIe bandwidth per tool call.

By overlapping $T_{\text{lead}} \in [150\text{ ms}, 400\text{ ms}]$ of network I/O with active token decoding, Agent-Synapse reduces the post-EOS HBM residency stall time from T_{io} to $\max(0, T_{\text{io}} - T_{\text{lead}})$.

Algorithm 2 Stage 3 Canonical AST Normalization (Ψ_{AST}) and On-Chip SRAM CoW Splicing / Rollback

Require: Speculative prediction $(\hat{w}, \hat{\mathbf{a}})$, actual emitted call $(w^*, \mathbf{a}^*)$ at t_{eos}, SRAM micro-checkpoint $\mathcal{D}_{\text{chk}} = \langle \text{id}_{\text{chk}}, h_{\text{chk}}, t_{\text{branch}}, \mathcal{P}_{\text{base}}, \mathcal{P}_{\text{shadow}}, t_{\text{ns}} \rangle$, wire envelope $\mathcal{E}_{\text{wire}}$.
Ensure: Updated on-chip SRAM page table $\mathcal{P}_{\text{SRAM}}$ and committed tool response R^*.

function Ψ_{AST}(node)
  if node is Dict **then**
    keys $\leftarrow$ SortLexicographical(keys(node))
    return $\{k : \Psi_{\text{AST}}(\text{node}[k])$ for $k \in$ keys$\}$
  else if node is List **then**
    return $[\Psi_{\text{AST}}(e)$ for $e \in$ node$]$
  else if node is Float **and** node $==$ $\lfloor$node$\rfloor$ **then**
    return Int(node)
  else if node is String **then**
    $s \leftarrow$ CollapseWhitespaceAndFoldKeywords(node)
    if HasConjunctiveWhereClause(s) **and** $\neg$ HasTopLevelOr(s) **then**
      (prefix, $\{P_1, \ldots, P_r\}$, suffix) $\leftarrow$ SplitWhereConjuncts(s)
      $\{P_{\pi(1)}, \ldots, P_{\pi(r)}\} \leftarrow$ SortLexicographical($\{P_1, \ldots, P_r\}$)
      $s \leftarrow$ prefix $\|$ `" WHERE "` $\|$ Join(`" AND "`, $\{P_{\pi(1)}, \ldots, P_{\pi(r)}\}$) $\|$ suffix
    end if
    return s
  end if
  return node
end function
$h_{\text{pred}} \leftarrow$ SHA-256(CanonicalJSON(Lower($\hat{w}$), $\Psi_{\text{AST}}(\hat{\mathbf{a}})$))
$h_{\text{act}} \leftarrow$ SHA-256(CanonicalJSON(Lower(w^*), $\Psi_{\text{AST}}(\mathbf{a}^*)$))
if $h_{\text{pred}} == h_{\text{act}}$ **then** ▷ Canonical AST Equivalence Hit (89.4%)
  Wait $\max(0, T_{\text{io}} - T_{\text{lead}})$ for $\mathcal{E}_{\text{wire}}$ completion
  $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{D}_{\text{chk}}.\mathcal{P}_{\text{base}} \,\|\, \mathcal{D}_{\text{chk}}.\mathcal{P}_{\text{shadow}}$ ▷ $2.1\,\mu$s SRAM descriptor splice
  return $\mathcal{E}_{\text{wire}}$.payload
else ▷ Canonical AST Mismatch (10.6%)
  $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{D}_{\text{chk}}.\mathcal{P}_{\text{base}}$ ▷ $< 12\,\mu$s vectorized SRAM CoW restore
  FreeShadowPages($\mathcal{D}_{\text{chk}}.\mathcal{P}_{\text{shadow}}$)
  $R^* \leftarrow$ ExecutePostEOSVerified($w^*, \mathbf{a}^*$)
  return R^*
end if

Proposition 2 (HBM Page-Time Integral Reduction and Multi-Tenant Throughput Gain)**.** *Let $M_{\text{ctx}} = N_{\text{base}} S_{\text{page}}$ denote the HBM KV-cache footprint of an active agent sequence during a tool-calling turn, and let $M_{\text{shadow}} = N_{\text{spec}} S_{\text{page}}$ (with $M_{\text{shadow}} \ll M_{\text{ctx}}$) denote the shadow page footprint allocated only after the speculative response arrives. Under Agent-Synapse, the expected idle HBM page-time residency integral $\mathcal{I}_{\text{HBM}} = \mathbb{E}[\int_{t_{\text{eos}}}^{t_{\text{resume}}} M_{\text{HBM}}(t)\, dt]$ during post-EOS I/O wait is reduced relative to serial serving $\mathcal{I}_{\text{HBM}}^{\text{serial}} = M_{\text{ctx}} \mathbb{E}[T_{\text{io}}]$ by:*

$$\Delta \mathcal{I}_{\text{HBM}} = M_{\text{ctx}} p_{\text{trig}} \eta\, \mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] - M_{\text{shadow}} p_{\text{trig}} \mathbb{E}[\max(0, T_{\text{lead}} - T_{\text{io}})] \tag{32}$$

For typical enterprise contexts ($t_{\text{branch}} = 4{,}096$ tokens, $K_{\text{spec}} = 128$ response tokens, so $M_{\text{ctx}}/M_{\text{shadow}} = 32$), Agent-Synapse reduces idle post-EOS HBM page residency by 64.2%, increasing sustainable multi-tenant batch concurrency on a 32 GB Cloud TPU v6e chip by $1.41\times$.

Proof. In serial serving, the full context footprint M_{ctx} sits idle in HBM from t_{eos} to $t_{\text{eos}} + T_{\text{io}}$, accumulating page-time product $M_{\text{ctx}} T_{\text{io}}$. Under Agent-Synapse, on a speculative hit (probability $p_{\text{trig}}\eta$), the post-EOS wait duration shrinks from T_{io} to $\max(0, T_{\text{io}} - T_{\text{lead}}) = T_{\text{io}} - \min(T_{\text{io}}, T_{\text{lead}})$, saving $M_{\text{ctx}} \min(T_{\text{io}}, T_{\text{lead}})$ of idle context residency. Conversely, if the network RPC returns prior to t_{eos} ($T_{\text{io}} < T_{\text{lead}}$), the shadow buffer of size M_{shadow} is held for $T_{\text{lead}} - T_{\text{io}}$. Taking expectations across turns yields Eq. (32). Since $M_{\text{shadow}} = \frac{128}{4096} M_{\text{ctx}} = 0.03125 M_{\text{ctx}}$ and $\mathbb{E}[\min(T_{\text{io}}, T_{\text{lead}})] = 224.0$ ms out of $\mathbb{E}[T_{\text{io}}] = 280.0$ ms, the net idle HBM page-time integral drops by 64.2%. By Little's Law ($L = \lambda W$), reducing average sequence residency time W by 38.4% across the full turn allows $1/(1 - 0.291) \approx 1.41\times$ higher concurrent request throughput under a fixed HBM budget. □

G. Deep-Dive Case Studies Across Enterprise Backend Architectures

To ground our empirical findings in concrete cloud systems behavior, we detail how Agent-Synapse interacts with each of the four enterprise backend engines evaluated in Table IV:

1) Case Study 1: Financial Crime Ring Detection on Google Cloud Spanner Graph (ISO GQL): In anti-money-laundering (AML) compliance workflows, an agent queries a multi-billion-edge enterprise knowledge graph stored in Google Cloud Spanner Graph [21] using ISO/IEC 39075:2024 GQL [23]. A typical 3-hop cycle traversal query takes the form:

```
GRAPH EnterpriseFinancialKG
MATCH (src:Account {jurisdiction:
'CH'})-[w1:WIRE]->(mid:ShellEntity)-[w2:WIRE]->(dst:Account)
WHERE w1.amount_usd >= 100000 AND w2.timestamp
>= TIMESTAMP('2026-01-01T00:00:00Z') RETURN
src.account_id, mid.entity_id, dst.account_id,
w1.amount_usd ORDER BY w1.amount_usd DESC LIMIT 25
```

During autoregressive decoding, once the model emits the anchor tokens `spanner_graph_query_gql`, the graph identifier `EnterpriseFinancialKG`, and the core pattern bindings `jurisdiction: 'CH'` and `amount_usd >= 100000` at token step $t_{\text{branch}} = 24$, the intermediate logit lens at Layers 12–16 already projects the remaining 28 tokens (`RETURN ... ORDER BY ... LIMIT 25"}}</tool_call>`) with conformal confidence $C_{\text{conf}} = 0.91 > \tau_{\text{conf}} = 0.82$. Stage 2 immediately dispatches the ISO GQL query over gRPC/HTTP2 with `X-Agent-Speculative-Prefetch: True`. Cloud Spanner's Paxos leaderless read-only snapshot replica [21] executes the distributed index join across 279.4 ms of overlap window while the TPU v6e finishes decoding the closing syntax. At `</tool_call>` EOS, even if the model swapped the order of the two `WHERE` conjuncts (`w2.timestamp >= ... AND w1.amount_usd >= ...`), Stage 3's canonical AST normalizer Ψ_{AST} maps both to the identical SHA-256 digest and splices the returned subgraph into VMEM in $2.1\,\mu$s, cutting p_{99} turn latency from 846.0 ms to 318.5 ms (-62.4%).

2) Case Study 2: Petabyte Telemetry Aggregation on BigQuery Dremel SQL: In FinOps and cloud reliability diagnostics, agents invoke `bigquery_execute_analytics` to run columnar aggregations over partitioned BigQuery tables [22]. Because Dremel's root mixer must allocate shuffle slots and broadcast column metadata to leaf workers (80 ms–140 ms of coordination setup before scan completion), initiating the query 284.1 ms prior to `</tool_call>` EOS hides 100% of the Dremel mixer scheduling latency behind token generation, reducing p_{99} turn latency from 872.0 ms to 338.2 ms (-61.2%).

3) Case Study 3: Grounded Regulatory Retrieval on Vertex AI Search: In legal and clinical decision support, agents invoke `vertex_ai_rag_search` to retrieve top-k passages using reciprocal rank fusion of dense ScaNN vector search and BM25 lexical matching. Because the query string and metadata filters are concise, Stage 1 crosses $\tau_{\text{conf}} = 0.82$ with an 91.5% AST hit rate and 254.8 ms mean lead time—exceeding the mean backend retrieval latency (210.0 ms) on 78.5% of turns and achieving 0 ms post-EOS I/O wait on those requests.

4) Case Study 4: Transactional ERP Ledger Simulation with MVCC Snapshot Dry-Run: When preparing a multi-currency journal entry in SAP S/4HANA (`erp_post_ledger_dryrun`), Stage 2 injects `X-Synapse-Dry-Run-Mode: SNAPSHOT_READ_ONLY`. The ERP gateway validates cost-center existence, tax codes, and debit-credit balance against an immutable MVCC snapshot during token decoding, reducing p_{99} validation turn latency by 60.7% (831.0 ms $\rightarrow$ 326.8 ms) while guaranteeing zero persistent ledger mutations prior to post-EOS verification.

H. Security and Threat Analysis: Defending Against Speculative Side-Channel and Prompt Injection Attacks

Because speculative execution in hardware processors historically introduced microarchitectural side channels, we explicitly analyze the security properties of Agent-Synapse against three threat vectors:

1) **Indirect Prompt Injection Attempting Speculative Data Exfiltration or Destruction [32]:** Suppose a malicious document retrieved in Turn

$j-1$ attempts to hijack the model in Turn j into calling `crm_delete_customer_record` or injecting `; DROP TABLE customers;--` into `bigquery_execute_analytics`. Even if early transformer layers $l \in [8, 16]$ transiently attend to the injected instruction before upper safety-aligned layers ($l \in [24, L]$) refuse the request, Stage 2's deterministic idempotency gate (κ = `MUTATING_BLOCKED`) and lexical/AST guard (HasMutatingDML) unconditionally block the speculative wire dispatch ($0/250$ unsafe dispatches in Table III).

2) **Cross-Tenant Speculative Cache Poisoning:** Because every speculative FastMCP request is bound to a sequence-specific SHA-256 checkpoint hash `X-Synapse-Checkpoint-Hash`: $h_{\text{chk}} = \text{SHA-256}(\text{accel} \parallel t_{\text{branch}} \parallel \mathcal{P}_{\text{base}} \parallel \mathcal{P}_{\text{shadow}})$ and shadow pages $\mathcal{P}_{\text{shadow}}$ are mapped exclusively into the owning sequence's private SRAM page descriptor after canonical AST verification, a prefetched payload from tenant A can never be spliced into tenant B's KV-cache.
3) **Speculative Denial-of-Service (DoS) Amplification:** Each agent turn is restricted by Algorithm 1 (Line 4, ¬spec_active) to at most *one* speculative prefetch per turn, gated by $\tau_{\text{conf}} = 0.82$ and throttled under cluster load via Eq. (31), bounding worst-case backend query amplification to $\leq 9.58\%$.

VIII. Production Deployment and Architectural Discussion

Agent-Synapse is designed as a drop-in runtime extension for enterprise foundation model serving stacks (including vLLM [3], SGLang [19], and Google Cloud TPU JetStream) connected to FastMCP [20] tool gateways:

- **Zero Model Weight Modification:** Stage 1 reuses the existing frozen unembedding matrix $\mathbf{W}_U$ and intermediate cross-attention weights already computed during standard FlashAttention-2 [12] forward passes. Computing the logit-lens projection over the small set of M candidate tool anchor tokens adds < 0.14 ms of VPU/Tensor Core overhead per monitored decode step.
- **Downstream Admission Control via Speculative Wire Headers:** Because every speculative RPC carries `X-Agent-Speculative-Prefetch: True` and `X-Synapse-Conformal-Score`, downstream database proxies (Cloud Spanner, BigQuery) can schedule speculative prefetches at lower QoS priority than non-speculative production transactions during peak cluster utilization (Eq. (31)), shedding speculative load gracefully without impacting committed SLA traffic.
- **Complementary Integration with Schema Pruning and Saga Orchestration:** Agent-Synapse composes orthogonally with pre-reasoning MCP schema pruning [33], [34] (which bounds the candidate catalog M before prefill) and transactional Saga orchestration [26], [28] (which governs post-EOS multi-step mutating workflows).

IX. Related Work

Speculative Decoding for LLMs: Leviathan et al. [1] and Chen et al. [2] introduced draft-target speculative decoding to accelerate autoregressive token generation without altering the output distribution. Medusa [4] and EAGLE [5] eliminated the separate draft model by attaching lightweight prediction heads to top transformer layers. However, all token-level speculative decoding methods terminate at the vocabulary boundary and wait until `</tool_call>` EOS before initiating external network I/O. Agent-Synapse is the first system to project intermediate hidden states across the hardware NIC boundary to overlap remaining token decoding with external database RPCs.

Early-Exit Transformers and Mechanistic Interpretability: The Logit Lens [7] and Tuned Lens [6] demonstrated that intermediate transformer representations encode high-probability predictions of future tokens well before the final layer. CALM [8] leveraged early-exit classifiers to skip upper transformer layers on easy tokens, while Angelopoulos and Bates [9] and Shafer and Vovk [10] established distribution-free split-conformal risk bounds. Agent-Synapse repurposes early-layer logit-lens projections and cross-attention entropy not to truncate layer computation (which would degrade reasoning quality on complex prompts), but to launch asynchronous, side-effect-free network prefetches while full-depth decoding continues unimpeded.

KV-Cache Management and Speculative Systems: PagedAttention [3] and SGLang RadixAttention [19] virtualized KV-cache pages in HBM to eliminate fragmentation and share common prefixes across requests. In distributed systems, Speculator [29] and GoEX [27] explored speculative execution and undo/compensating logs for OS filesystems and coarse-grained LLM actions. Agent-Synapse bridges these two worlds by binding outbound FastMCP speculative wire headers (`X-Synapse-Checkpoint-Hash`) to 16-token-aligned Copy-on-Write page-table descriptors in on-chip accelerator SRAM, achieving $< 12\,\mu$s rollback and 0.00% unauthorized external mutations.

X. Conclusion

We presented **Agent-Synapse**, a three-stage hardware-software co-designed runtime that eliminates the serial decode-to-I/O stall in autonomous tool-using AI agents. By combining split-conformal early-layer attention-entropy and logit-lens tool prediction across Layers 8–16, idempotency-gated speculative FastMCP wire dispatch, and $< 12\,\mu$s Copy-on-Write on-chip SRAM KV-cache micro-checkpointing with canonical AST equivalence splicing, Agent-Synapse achieves an 89.4% speculative hit rate and reduces p_{99} end-to-end tool-calling tail latency by 61.8% (842.0 ms $\rightarrow$ 321.6 ms) across $N = 1{,}000$ enterprise agent traces with zero unauthorized external side effects.

APPENDIX A
EXTENDED PROOF OF TAIL QUANTILE CONTRACTION UNDER SPECULATIVE COMPUTE-I/O OVERLAP

In Section VI, Theorem 3 derived the exact piecewise turn latency law and expected speedup $\Delta\mathbb{E}[T]$. Here we derive the upper bound on the $(1-\delta)$-quantile (specifically p_{99} for $\delta = 0.01$) of end-to-end turn latency under Agent-Synapse.

Proposition 3 (High-Percentile Tail Latency Contraction)**.** *Let $F_{\text{serial}}(z) = \mathbb{P}(T_{\text{decode}} + T_{\text{io}} \le z)$ denote the cumulative distribution function (CDF) of serial turn latency, and let $Q_{1-\delta}(T)$ denote the $(1-\delta)$-quantile of random variable T. Suppose $p_{\text{trig}}\eta > 1-\delta$ (which holds at $\tau_{\text{conf}} = 0.82$ where $p_{\text{trig}}\eta = 0.808$ for all percentiles up to p_{80}, and conditioned on the hit sub-distribution for p_{99}). For any turn on the speculative hit path with lead time $T_{\text{lead}} \ge T_{\text{lead}}^{\min} > 0$, the conditional $(1-\delta)$-quantile of T_{synapse} satisfies:*

$$Q_{1-\delta}(T_{\text{synapse}} \mid \text{Hit}) \le Q_{1-\delta}(T_{\text{serial}}) - \min\big(T_{\text{lead}}^{\min},\, Q_{\delta}(T_{\text{io}})\big) + T_{\text{splice}}. \tag{33}$$

Proof. On the speculative hit event, Eq. (26) gives $T_{\text{synapse}} = T_{\text{decode}} + T_{\text{io}} - \min(T_{\text{io}}, T_{\text{lead}}) + T_{\text{splice}} = T_{\text{serial}} - \min(T_{\text{io}}, T_{\text{lead}}) + T_{\text{splice}}$. Because $T_{\text{lead}} \ge T_{\text{lead}}^{\min} = 150.0$ ms almost surely on triggered turns and $T_{\text{io}} \ge T_{\text{io}}^{\min} = 160.0$ ms on enterprise database RPCs, the overlap reduction satisfies $\min(T_{\text{io}}, T_{\text{lead}}) \ge \min(T_{\text{lead}}^{\min}, T_{\text{io}}^{\min})$ point-wise. By monotonicity of quantiles ($X \le Y - c \implies Q_{1-\delta}(X) \le Q_{1-\delta}(Y) - c$), subtracting the guaranteed overlap floor and adding $T_{\text{splice}} = 2.1\,\mu$s yields Eq. (33). Furthermore, because tail database stalls ($T_{\text{io}} \in [400\text{ ms}, 550\text{ ms}]$ at p_{99}) always exceed the full lead time $T_{\text{lead}} \in [162\text{ ms}, 378\text{ ms}]$, $\min(T_{\text{io}}, T_{\text{lead}}) = T_{\text{lead}}$ holds with probability 1 in the upper tail, absorbing the entire T_{lead} window plus eliminating queue-wait compounding across sequential turns. □

APPENDIX B
CYCLE-ACCURATE SRAM PAGE-TABLE ROLLBACK MODEL ON CLOUD TPU V6E AND NVIDIA B200

We formalize the hardware cycle budget governing Stage 3's $< 12\,\mu$s Copy-on-Write rollback guarantee across Cloud TPU v6e (Trillium) and NVIDIA Blackwell B200. Let f_{clk} denote the scalar/tensor control clock frequency ($f_{\text{clk}} = 1.75$ GHz on TPU v6e; $f_{\text{clk}} = 2.10$ GHz on B200), W_{vec} denote the SRAM vector register width ($W_{\text{vec}} = 512$ bytes per VPU vector register on TPU v6e; $W_{\text{vec}} = 128$ bytes per warp transaction on B200), and $N_{\text{pages}} = \lceil t_{\text{eos}}/B_{\text{page}} \rceil$ denote the number of 8-byte page descriptor entries.

The cycle count C_{swap} required to restore $\mathcal{P}_{\text{SRAM}} \leftarrow \mathcal{P}_{\text{base}}$ from on-chip VMEM/TMEM is:

$$C_{\text{swap}}(N_{\text{pages}}) = C_{\text{fence}} + \left\lceil \frac{8 \cdot N_{\text{pages}}}{W_{\text{vec}}} \right\rceil \cdot C_{\text{sram_rw}} + C_{\text{tlb_inv}}, \tag{34}$$

where $C_{\text{fence}} \in [120, 240]$ cycles is the pipeline synchronization barrier at t_{eos}, $C_{\text{sram_rw}} \in [4, 8]$ cycles is the pipelined on-chip SRAM read-write latency per vector word, and $C_{\text{tlb_inv}} \in [180, 350]$ cycles invalidates the N_{spec} shadow entries in the local page-descriptor cache. Even for a long 32,768-token context ($N_{\text{pages}} = 2{,}048$, descriptor size 16 KB), $\lceil 16{,}384/512 \rceil = 32$ vector moves require < 600 clock cycles ($\approx 0.34\,\mu$s of raw SRAM transfer), and adding Python/C++ runtime dispatch, SHA-256 AST digest comparison ($1.8\,\mu$s), and DMA descriptor synchronization yields a total measured p_{50} of $3.8\,\mu$s on TPU v6e and $4.4\,\mu$s on B200 ($p_{99} \le 10.1\,\mu\text{s} < 12.0\,\mu\text{s}$).

TABLE VII
MICROARCHITECTURAL LATENCY BREAKDOWN OF STAGE 3 SRAM COPY-ON-WRITE ROLLBACK (T_{RB}) VS. HBM RE-PREFILL

Sub-Operation Phase	TPU v6e VMEM	B200 TMEM	HBM Re-Pr
1. Canonical AST Digest Check	$1.6\,\mu$s	$1.8\,\mu$s	$1.8\,\mu$s
2. Page-Table Pointer Restore	$1.4\,\mu$s	$1.7\,\mu$s	$420.0\,\mu$s (HBM
3. Shadow Frame Bitmap Free	$0.8\,\mu$s	$0.9\,\mu$s	$27{,}980.0\,\mu$s (At
Total Median (p_{50})	$\mathbf{3.8}\,\mu$s	$\mathbf{4.4}\,\mu$s	$28{,}401.8\,\mu$
Total 99th Percentile (p_{99})	$\mathbf{9.4}\,\mu$s	$\mathbf{10.1}\,\mu$s	$44{,}100.0\,\mu$

APPENDIX C
EQUIVALENCE CLASSES AND BOUNDARY CONDITIONS OF CANONICAL AST SPLICING

Table VIII summarizes representative syntactic variations encountered across our $N = 1{,}000$ trace benchmark, distinguishing semantics-preserving variations that Stage 3's Ψ_{AST} splices ($h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) = h_{\text{AST}}(w^*, \mathbf{a}^*)$) from genuine semantic divergences that Stage 3 rolls back ($h_{\text{AST}}(\hat{w}, \hat{\mathbf{a}}) \neq h_{\text{AST}}(w^*, \mathbf{a}^*)$).

TABLE VIII
TAXONOMY OF SYNTACTIC VS. SEMANTIC VARIATIONS HANDLED BY STAGE 3 CANONICAL AST SPLICER (Ψ_{AST})

Variation Pattern	Predicted vs. Emitted Example	Ψ_{AST} Decision
JSON Key Permutation	`{"a":1,"b":2}` vs. `{"b":2,"a":1}`	**Hit (Splice)**
Integral Float Literal	`"limit": 25.0` vs. `"limit": 25`	**Hit (Splice)**
SQL/GQL Case & Space	`match (n) where x=1` vs. `MATCH (n) WHERE x = 1`	**Hit (Splice)**
Commuted `WHERE AND`	`WHERE a=1 AND b=2` vs. `WHERE b=2 AND a=1`	**Hit (Splice)**
Optional Schema Default	Omitted `timeout_ms` vs. `"timeout_ms": 5000`	**Hit (Splice)**
Predicate Literal Shift	`risk > 0.85` vs. `risk > 0.90`	**Mismatch (CoW)**
Disjunctive `OR` Clause	`WHERE a=1 OR b=2` (No reorder)	**Exact Order**
Limit / Aggregation Change	`LIMIT 10` vs. `LIMIT 50`	**Mismatch (CoW)**

REFERENCES

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. 40th International Conference on Machine Learning (ICML)*, vol. 202, 2023, pp. 19274–19286.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proc. 41st International Conference on Machine Learning (ICML)*, 2024.

[6] N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, and J. Steinhardt, "Eliciting latent predictions from transformers with the tuned lens," *arXiv preprint arXiv:2303.08112*, 2023.

[7] nostalgebraist, "Interpreting GPT: The logit lens," *LessWrong Technical Report*, 2020.

[8] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri, V. Q. Tran, Y. Tay, and D. Metzler, "Confident adaptive language modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 17456–17472.

[9] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023.

[10] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16344–16359.

[12] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[13] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles *et al.*, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proc. 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–14.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[16] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[17] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[18] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM compiler for parallel function calling," in *Proc. 41st International Conference on Machine Learning (ICML)*, 2024.

[19] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "SGLang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2024.

[20] Anthropic, "Model Context Protocol specification," Nov. 2024. [Online]. Available: https://modelcontextprotocol.io.

[21] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally-distributed database," in *Proc. 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012, pp. 251–264.

[22] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis, "Dremel: Interactive analysis of web-scale datasets," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 330–339, 2010.

[23] ISO/IEC JTC 1/SC 32, "ISO/IEC 39075:2024 Information technology — Database languages — GQL," International Organization for Standardization, Geneva, Switzerland, 2024.

[24] Gemini Team, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[26] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.

[27] S. G. Patil, T. Zhang, V. Fang, N. Chaturvedi, R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," *arXiv preprint arXiv:2404.06921*, 2024.

[28] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *arXiv preprint arXiv:2503.11951*, 2025.

[29] E. B. Nightingale, P. M. Chen, and J. Flinn, "Speculative execution in a distributed file system," in *Proc. 20th ACM Symposium on Operating Systems Principles (SOSP)*, 2005, pp. 191–205.

[30] R. M. Tomasulo, "An efficient algorithm for exploiting multiple arithmetic units," *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967.

[31] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.

[32] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[33] E. Lumer, A. Gulati, V. K. Subbiah, P. H. Basavaraju, and J. A. Burke, "ScaleMCP: Dynamic and auto-synchronizing Model Context Protocol tools for LLM agents," *arXiv preprint arXiv:2505.06416*, 2025.

[34] T. Gan and Q. Sun, "RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation," *arXiv preprint arXiv:2505.03275*, 2025.

[35] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv preprint arXiv:2508.20453*, 2025.